Universidad Nacional de Educación a Distancia
Escuela Técnica Superior de Ingeniería Informática

Proyecto de fin de Grado en Ingeniería Informática

IDE para desarrollo de aplicaciones para máquinas de 8 bits con C

Andrés Giménez Perales
Dirigido por: Jose Manuel Diaz Martinez

Curso 2025/2026, convocatoria Junio

IDE para desarrollo de aplicaciones para máquinas de 8 bits con C

Proyecto de fin de Grado en Ingeniería Informática

de modalidad Específica

Realizado por: Andrés Giménez Perales

Dirigido por: Jose Manuel Diaz Martinez

Fecha de lectura y defensa: Junio 2026

Agradecimientos

Quiero expresar mi más sincero agradecimiento a mi tutor, José Manuel Díaz Martínez, por su orientación, paciencia y apoyo constante durante la realización de este trabajo. Sus consejos y su rigor académico han sido fundamentales para el desarrollo de este proyecto.

Asimismo, agradezco al profesorado del Grado en Ingeniería Informática de la UNED por los conocimientos transmitidos a lo largo de estos años, que han hecho posible la elaboración de este estudio.

Finalmente, quiero agradecer a mi familia y a las personas que me han acompañado durante esta etapa por su apoyo incondicional, comprensión y motivación constante. A mis padres, por creer siempre que podría lograrlo; a mi mujer, por apoyarme sin condiciones; a mi hermano, por darme ejemplo; y muy especialmente a mi hija Aitana, por ser mi mayor motivación incluso en los momentos más difíciles.

Resumen

Este trabajo aborda el desarrollo de un entorno de desarrollo integrado (IDE) enfocado al desarrollo de software en C para máquinas de 8 bits. El objetivo principal es que la aplicación soporte el desarrollo para múltiples plataformas de ordenadores y videoconsolas de las décadas de 1980 y 1990 basadas en tecnología de 8 bits. Para ello, se ha desarrollado un software utilizando principalmente tecnologías Java, capaz de compilar código mediante distintos compiladores dirigidos a estas arquitecturas y que, además, incorpora un sistema de plugins que permite ampliar y extender sus funcionalidades.

Los resultados muestran que la herramienta proporciona un entorno unificado que simplifica el flujo de trabajo en el desarrollo para arquitecturas de 8 bits, mejorando la accesibilidad, la organización y la eficiencia del proceso.

Abstract

This work addresses the development of an Integrated Development Environment (IDE) focused on C software development for 8-bit systems. The main objective is to enable the application to support development for multiple platforms of computers and video game consoles from the 1980s and 1990s based on 8-bit technology. To this end, a software solution has been developed primarily using Java technologies, capable of compiling code through different compilers targeting these architectures and incorporating a plugin system that allows the extension and enhancement of its functionalities.

The results show that the tool provides a unified environment that simplifies the workflow in 8-bit architecture development, improving accessibility, organization, and overall process efficiency.

Palabras clave

IDE, desarrollo de software, lenguaje C, sistemas de 8 bits, compilación cruzada, arquitectura de plugins, ZX Spectrum, Game Boy.

Keywords

IDE, software development, C language, 8-bit systems, cross-compilation, plugin architecture, ZX Spectrum, Game Boy.

Índice

Índice de tablas XV

Índice de figuras XVIII

1. Introducción **1**

1.1. Motivación 1

1.2. Objetivos 2

1.3. Estado del arte 3

1.3.1. GBDK-2020 3

1.3.2. Z88DK 5

1.3.3. ZX Basic Studio 7

1.3.4. VS Code + plugins 8

1.3.5. Eclipse CDT + toolchains 10

1.4. Estructura de la memoria 12

2. Análisis **13**

2.1. Introducción 13

2.2. Objetivos y funciones 14

2.3. Requisitos mínimos y restricciones 15

2.4. Modelo de desarrollo utilizado 15

2.5. Tecnologías 16

2.5.1. Introducción 16

2.5.2. Java 16

2.5.3. Maven 17

2.5.4. GitHub 17

2.5.5. JavaFX 17

2.5.6. IntelliJ IDEA 18
2.5.7. Emulicious 19
2.5.8. JSpeccy 20
2.6. Descripción del modelo 21
2.6.1. Requisitos 21
2.6.2. Casos de uso 22
2.6.3. Casos de uso 25
2.6.4. Diagrama de estados 28
2.6.5. Diagrama de clases 29

3. Diseño **31**
3.1. Diseño del sistema 31
3.1.1. Arquitectura 31
3.1.2. Diseño de vistas 31

4. Desarrollo del proyecto **37**

5. Pruebas **39**

6. Conclusiones **41**

Bibliografía **43**

A. Manual de usuario **45**
A.1. Introducción 45
A.1.1. Características principales 45
A.2. Instalación 46
A.2.1. Requisitos del Sistema 46
A.2.2. Instalación desde Código Fuente 46
A.2.3. Instalación desde Ejecutable 46

A.3. Interfaz 47
A.3.1. Descripción de la Interfaz 47
A.4. Gestión de Proyectos 51
A.4.1. Crear un Nuevo Proyecto 51
A.4.2. Abrir un Proyecto Existente 52
A.4.3. Trabajar con Archivos 52
A.5. Edición de Código 53
A.5.1. Resaltado de Sintaxis 53
A.5.2. Operaciones de Edición 54
A.5.3. Características Avanzadas 54
A.6. Búsqueda de Texto 55
A.6.1. Buscar en Archivo Actual 55
A.6.2. Buscar en Todo el Proyecto 56
A.7. Miscelánea 57
A.7.1. Atajos de Teclado 57
A.7.2. Tecnologías utilizadas en el desarrollo de SamaruC Retro Editor: 57
A.7.3. Requisitos del Sistema: 58

B. Desarrollo de plugins **59**
B.1. Introducción 59
B.2. Arquitectura general 59
B.3. Requisitos previos 60
B.4. Crear el proyecto Maven del plugin 61
B.5. Implementar la interfaz `Plugin` 62
B.6. Registrar el plugin con ServiceLoader 63
B.7. Usar el `PluginContext` 64
B.7.1. Ejemplo: abrir un diálogo con el Stage del host 64

B.7.2. Ejemplo: escribir en el Monitor 65
B.8. Empaquetar el plugin como JAR 65
B.9. Instalar el plugin en SamaruC 65
B.9.1. Opción A — Desde la interfaz de SamaruC (recomendado) 65
B.9.2. Opción B — Copiado manual 66
B.10. Habilitar y deshabilitar plugins 66
B.11. Ejemplo completo 67
B.11.1. Estructura del proyecto 67
B.11.2. `UtilsPlugin.java` 67
B.11.3. `META-INF/services/com.retroeditor.plugin.Plugin` 68
B.11.4. Compilar e instalar 68
B.12. Buenas prácticas y limitaciones 69
B.12.1. Buenas prácticas 69
B.12.2. Limitaciones actuales 69

Índice de tablas

1.1. Comparativa de herramientas y entornos para desarrollo en plataformas de 8 bits 3

1.2. Selección representativa de plataformas soportadas por el compilador z88dk. Adaptado de la documentación oficial [1]. 6

1.3. Plugins de VS Code para desarrollo en plataformas de 8 bits 9

1.4. Características clave de Eclipse CDT para desarrollo en 8 bits 11

A.1. Atajos de teclado disponibles en SamaruC Retro Editor. 57

B.1. Ciclo de vida de un plugin 60

B.2. Métodos de PluginContext 64

Índice de figuras

1.1. Captura de pantalla del proyecto Orange FM. 4

1.2. Captura de pantalla del proyecto ZNAKE. 6

1.3. Captura de pantalla del proyecto ZX Basic Studio. 7

1.4. Captura de pantalla de Visual Studio Code. 8

1.5. Captura de pantalla de Eclipse CDT. 10

2.1. Lenguajes soportados por IntelliJ IDEA según el libro de Hagos (Beginning IntelliJ IDEA)[2]. 18

2.2. Interfaz del debugger integrado de Emulicious mostrando las herramientas de desarrollo disponibles. 19

2.3. Interfaz de JSpeccy y sus diferentes opciones de configuración. 20

2.4. Diagrama de casos de uso del sistema de desarrollo IDE. 24

2.5. Diagrama de estados del entorno de desarrollo. 28

2.6. Diagrama de clases del sistema. 29

3.1. Diálogo de configuración de idioma. 32

3.2. Diálogo de configuración del compilador GBDK-2020. 33

3.3. Diálogo de configuración del compilador Z88DK. 34

3.4. Diálogo de configuración de plugins. 35

3.5. Diálogo para la creación de nuevos proyectos. 35

3.6. Diálogo de búsqueda en archivos. 35

3.7. Diálogo de búsqueda a nivel de proyecto. 36

3.8. Diálogo para ir a una línea específica. 36

3.9. Interface de configuración del emulador JSpeccy. 36

A.1. Situación de los menus desplegables de la herramienta. 47

A.2. Barra de herramientas con los iconos representativos de cada función. 48

A.3. Panel del explorador de proyectos. 49
A.4. Área de edición con pestañas. 50
A.5. Panel de consola y monitor. 50
A.6. Barra de estado. 50
A.7. Crear un nuevo proyecto. 51
A.8. Abrir un proyecto existente. 52
A.9. Resaltado de sintaxis en el editor de código. 53
A.10. Búsqueda en proyecto actual. 56
A.11. Resultados de búsqueda en proyecto. 56

B.1. Diagrama de arquitectura del sistema de plugins. 59
B.2. Estructura de directorios del proyecto Maven del plugin. 61

Capítulo 1

Introducción

1.1. Motivación

En las ciencias de la computación, las tecnologías avanzan rápidamente, y los sistemas antiguos suelen quedar obsoletos; sin embargo, ocasionalmente surge la necesidad de recuperar el uso de máquinas históricas, motivados por el interés histórico o educativo.

Durante la segunda mitad de los años 80 y la primera de los 90, la industria del entretenimiento experimentó un notable crecimiento debido a la proliferación de fabricantes de hardware a nivel mundial. Ordenadores domésticos como el ZX Spectrum o el Amstrad CPC llegaron a los hogares europeos, mientras que videoconsolas de 8 bits, basadas en procesadores como el Z80 o el 68000 y procedentes de Asia, permitieron que muchos usuarios tuvieran sus primeros contactos con software desarrollado, como la Nintendo Entertainment System o la portátil Game Boy.

El desarrollo de software para estas plataformas requiere conocimientos específicos de sus arquitecturas y limitaciones. En aquella época, los recursos eran reducidos y la mayoría del software se programaba en ensamblador directamente para el procesador objetivo, sin herramientas modernas de desarrollo.

En la actualidad, aunque existen compiladores para estas máquinas, su uso requiere gestionar herramientas dispersas y un conocimiento avanzado del entorno, lo que limita la accesibilidad y aumenta la complejidad del desarrollo.

El desarrollo enfocado a estas plataformas, además tiene ventajas formativas de cara a la preparación de futuros desarrolladores ya que centra el desarrollo de software en la optimización de recursos disponibles y la programación cercana al hardware.

Otro aspecto relevante es la preservación del patrimonio cultural digital. Aunque actualmente puede encontrarse información sobre sistemas históricos en internet, esta se halla dispersa y depende en gran medida de iniciativas individuales y voluntarias. La falta de centralización y de mecanismos formales de conservación implica el riesgo de que, a medio o largo plazo, parte del conocimiento técnico asociado a estas arquitecturas, así como el software desarrollado para ellas, pueda desaparecer. En este sentido, el presente proyecto contribuye a la preservación de esta cultura tecnológica mediante la promoción de la reutilización de estas plataformas, el desarrollo de nuevo software y la adaptación parcial de aplicaciones a entornos actuales.

1.2. Objetivos

El objetivo de este trabajo es crear un entorno de desarrollo integrado (IDE) en Java que permita el desarrollo de software en C para sistemas de 8 bits. Entre sus características principales se incluyen la compilación cruzada, un sistema de plugins para ampliar funcionalidades y la posibilidad de añadir nuevos compiladores.

Objetivos específicos:

- Implementar integración automática con compiladores cruzados.
- Diseñar una arquitectura modular basada en plugins.
- Permitir la incorporación de nuevas plataformas sin modificar el núcleo del sistema.
- Simplificar el flujo de trabajo del desarrollador de software clásico.

El alcance del proyecto se centra principalmente en las plataformas Game Boy y ZX Spectrum, dado que existen librerías ya desarrolladas (GBDK [3] y Z88DK [4]) y el software para estas máquinas es de pequeño tamaño, facilitando la realización de pruebas y la validación de la herramienta. Otra gran ventaja de usar estos sistemas es que utilizan arquitecturas basadas total o parcialmente en el procesador Z80 desarrollado por Zilog. Asimismo, se ha diseñado el sistema de plugins para permitir futuras ampliaciones del IDE y facilitar la inclusión de más sistemas desarrollables dentro de la herramienta.

Justificación técnica de la selección de plataformas Gameboy y ZX Spectrum:

- Ambas usan CPU basadas en Z80 (total o parcialmente).
- Comunidad activa.
- Librerías existentes.
- Tamaño reducido de ROM → validación rápida.

Los capítulos siguientes presentan la metodología utilizada, el desarrollo de la herramienta, las pruebas realizadas, los resultados obtenidos y las conclusiones derivadas de este proyecto.

1.3. Estado del arte

En este apartado se analizarán los ejemplos más representativos de herramientas para el desarrollo de aplicaciones de 8 bits. Se incluirán tanto IDEs como compiladores.

La Tabla 1.1 recoge un resumen comparativo de las principales características de los entornos de desarrollo más representativos que existen en la actualidad, que serán analizadas en detalle en las subsecciones posteriores. Se prestará especial atención a distinguir entre aquellas funcionalidades integradas de forma nativa y aquellas que dependen de complementos externos o configuraciones adicionales.

Característica	**GBDK**	**Z88DK**	**ZX Basic Studio**	**VS Code + plugins**	**Eclipse CDT + toolchains**
IDE integrado	No	No	Sí	Sí	Sí
Edición de código avanzada	No / mín.	No / mín.	Sí	Sí	Sí
Compilación cruzada C	Sí	Sí	No	Sí*	Sí*
Soporte múltiples plataformas	No (GB)	Parcial (Z80)	No (ZX Spectrum)	Sí*	Sí*
Depuración integrada	No	No	Sí	Sí*	Sí*
Integración con emuladores	No	No	Sí	Sí*	Sí*
Arquitectura de plugins/extensible	No	No	No	Sí*	Sí*
Curva de aprendizaje	Alta	Alta	Baja-media	Media-alta	Alta
Orientado específicamente a 8 bits	Sí	Sí	Sí	Sí*	Sí*
Enfoque formativo	Medio	Medio	Medio	Variable	Variable
Nota: El asterisco (*) indica que la característica depende de configuración manual.					

Tabla 1.1: Comparativa de herramientas y entornos para desarrollo en plataformas de 8 bits

1.3.1. GBDK-2020

Como se indica en la documentación oficial, GBDK es un kit de desarrollo cruzado para consolas basadas en SM83, Z80 y 6502. Incluye bibliotecas, utilidades de toolchain y el conjunto de compiladores C SDCC [3].

En la actualidad, el proyecto se escindió en una rama diferente llamada GBDK-2020 y el proyecto original, aunque aún es posible encontrarlo en internet, no está mantenido por sus autores Pascal Felber, Lars Malmborg, Michael Hope, David Galloway (djmips), John Fuge y colaboradores.

Incluye tanto las librerías necesarias como el compilador para usarlas además de documentación completa.

Un ejemplo de software desarrollado usando este kit es el proyecto Orange FM, cartucho moderno diseño para la familia de consolas portátiles Game Boy que funciona como un sistema de radio estéreo [5]. En la Figura 1.1 se muestra la interfaz de este proyecto ejecutándose en la consola Game Boy.

Características de GBDK-2020:

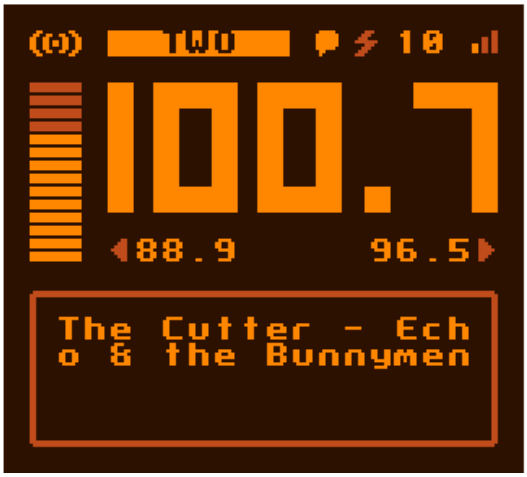

Figura 1.1: Captura de pantalla del proyecto Orange FM.

- Conjunto de herramientas para desarrollo en C y ASM basada en SDCC.
- Múltiples bibliotecas y su código fuente.
- Programas de ejemplo realizados en ASM y en C.
- Soporte para múltiples bancos de memoria ROM.
- La mayor parte del código de las herramientas está bajo la licencia GNU GPL.
- GBDK-2020 permite el desarrollo para las siguientes plataformas
  - Nintendo Game Boy / Game Boy Color.
  - Analogue Pocket.
  - Sega Master System y Game Gear.
  - Mega Duck / Cougar Boy.
  - NES.

Además incluye diversas utilidades para facilitar los desarrollos:

- GBCompress Comprime (y descomprime) datos de archivos binarios con el algoritmo gbcompress.
- png2asset Herramienta para convertir archivos PNG a MetaSprites y mapas de mosaicos en formato GBDK.
- Romusage Utilidad para estimar el uso de ROM de Game Boy y SMS/GG a partir de archivos .noi y .map, ROM binarias y más.

Todo lo anteriormente comentado convierte GBDK-2020 en entorno de desarrollo multiplataforma para sistemas retro de 8 bits, pero sigue dependiendo de un IDE externo o la combinación de editor + terminal para el desarrollo de software.

1.3.2. Z88DK

Z88DK es un kit de desarrollo cruzado para sistemas basados en Z80. Incluye bibliotecas, utilidades de toolchain y el conjunto de compiladores C SDCC.

Fue creado originalmente por Dominic Morris a finales de la década de 1990 con el objetivo de facilitar el desarrollo de software en lenguaje C para el ordenador Cambridge Z88. Con el tiempo, el proyecto ha evolucionado hasta convertirse en un entorno de desarrollo multiplataforma mantenido por una comunidad de desarrolladores que amplían continuamente el soporte para diferentes sistemas basados en procesadores Z80 [4].

Como se indica en la documentación oficial: z88dk es el único kit de desarrollo de ensamblador y C que viene listo para usar para crear programas para más de 100 máquinas de la familia z80 (8080, 8085, gbz80, z80, z180, ez80_z80, KC160, Rabbit 2000, 3000, 4000, 5000) [4].

El compilador z88dk soporta un gran número de sistemas basados en procesadores Z80 y arquitecturas relacionadas. La tabla 1.2 muestra una selección representativa de algunas de las plataformas más conocidas soportadas por este entorno de desarrollo.

Algunas de las herramientas incluidas en Z88DK:

- zcces la interfaz de la cadena de herramientas. zcc puede generar un binario de salida a partir de cualquier conjunto de archivos fuente de entrada.
- z88dk-scc80 es el compilador C nativo de z88dk.
- z88dk-zsdcc es la personalización de z88dk del compilador C optimizado de sdcc.
- z88dk-z80asm es un ensamblador/vinculador/bibliotecario con todas las funciones que implementa secciones.
- z88dk-zobjcopy permite manipular archivos de objetos y bibliotecas creados por Z80ASM.
- z88dk-appmake procesa los binarios generados por el kit de herramientas en un formato adecuado para equipos de destino específicos. Por ejemplo, puede generar archivos hexadecimales de Intel, cintas, ROM, imágenes de disco, etc.
- z88dk-lib es un instalador de bibliotecas de terceros. Gestiona la instalación, la eliminación y el listado de bibliotecas disponibles.
- z88dk-zx0 y z88dk-zx7 son herramientas de compresión de datos.

ZNAKE [6] es un ejemplo de software desarrollado para ZX Spectrum programado en C y compilado con z88dk y compatible con versiones recientes de este compilador, fué publicado en itch.io por marco's retrobits. La Figura 1.2 ilustra una captura de pantalla del juego en ejecución en un emulador de ZX Spectrum.

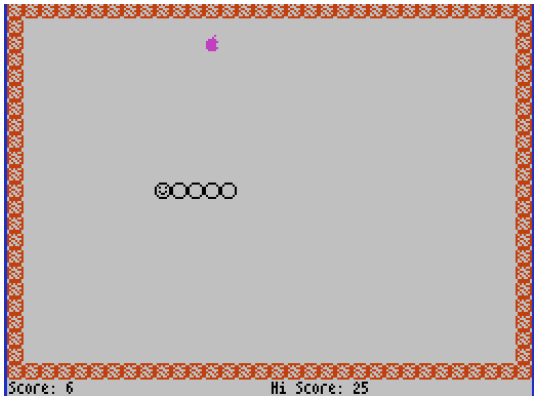

Figura 1.2: Captura de pantalla del proyecto ZNAKE.

Machine	Console	Graphics	File I/O	Sound
Altair 8800	Yes	No	No	No
Amstrad CPC	Yes	640x200	Yes	PSG
Apple II	40x24	No	CP/M	No
Cambridge Z88	Yes	256x64	Yes	1 bit
Colecovision	32x24	256x192	No	PSG
Commodore 128 (Z80)	40x25	640x200	CP/M	SID + PSG
Enterprise 64/128	40x25	672x243	No	Yes
Jupiter Ace	32x24	64x48	No	1 bit
MSX	40x24	256x192	No	PSG
NEC PC-8801	40x25	600x200	No	PSG
Nintendo Game Boy	20x18	160x144	No	Yes
Sega Master System	32x24	256x192	No	PSG
Sharp X1	40x25	640x200	No	PSG
Sinclair ZX80	32x24	64x48	No	1 bit
Sinclair ZX81	32x24	64x48	No	1 bit
ZX Spectrum	32x24	256x192	Yes	1 bit + PSG
ZX Spectrum Next	32x24	256x192	Yes	1 bit + PSG
SAM Coupé	32x24	512x192	No	PSG
TRS-80	64x16	128x48	No	1 bit
TI-83+	16x8	128x64	No	1 bit

Tabla 1.2: Selección representativa de plataformas soportadas por el compilador z88dk. Adaptado de la documentación oficial [1].

1.3.3. ZX Basic Studio

ZX Basic Studio [7] es un proyecto en desarrollo, iniciado por un desarrollador que utiliza el seudónimo "Dr. Gusman". Se trata de un IDE multiplataforma orientado al compilador Boriel Basic [8], un compilador BASIC de código abierto dirigido a usuarios de ZX Spectrum. Boriel Basic permite escribir programas en BASIC que se compilan a código máquina eficiente, ofreciendo facilidad de uso y rendimiento.

El IDE ZX Basic Studio está disponible para Windows, Linux y Mac. Integra un emulador y un depurador, así como un editor de gráficos, basado en una versión mejorada de ZX Graphics desarrollada por Juan Segura Duran ("Duefectu") [9]. Además, incluye un editor de ficheros .tap que permite gestionar no solo archivos individuales, sino proyectos completos. Su sistema de plugins amplía las funcionalidades del IDE, incorporando editores de gráficos, coloreadores de sintaxis, tooltips de ayuda y otras herramientas de soporte.

Actualmente, ZX Basic Studio permite compilar y ejecutar programas para ZX Spectrum Next, aunque por el momento la depuración directa en hardware real no está soportada. Dentro del ecosistema ZX Spectrum, ZX Basic Studio se posiciona como una herramienta moderna que combina edición de código, depuración y soporte gráfico, ofreciendo una alternativa más integrada frente a otros IDE tradicionales o emuladores con funcionalidades limitadas. En la Figura 1.3 se puede apreciar la interfaz del IDE con sus diferentes paneles y herramientas integradas.

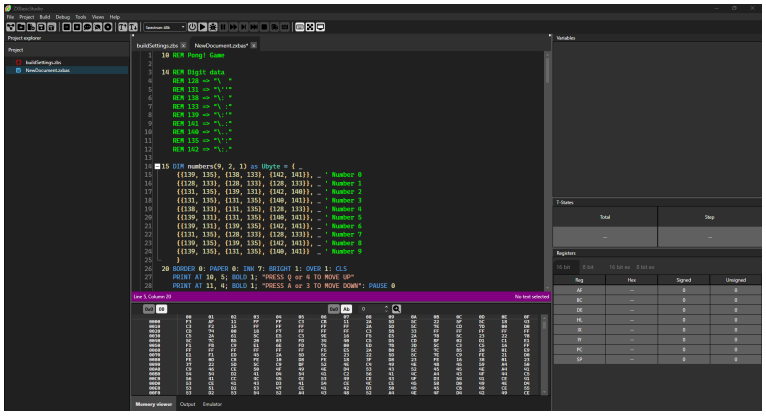

Figura 1.3: Captura de pantalla del proyecto ZX Basic Studio.

1.3.4. VS Code + plugins

Visual Studio Code [10] combina la simplicidad de un editor de código fuente con herramientas avanzadas para desarrolladores, como autocompletado de código y depuración mediante IntelliSense. Su diseño minimalista permite que el flujo de edición, compilación y depuración sea ágil, facilitando la concentración en el desarrollo de software.

VS Code es un editor rápido, adecuado para un uso intensivo. Ofrece compatibilidad con la mayoría de los lenguajes de programación, incluyendo resaltado de sintaxis, coincidencia de corchetes, sangría automática, selección de columnas y fragmentos de código reutilizables.

Incluye un depurador interactivo que permite recorrer el código fuente paso a paso, inspeccionar variables, consultar la pila de llamadas y ejecutar comandos en la consola. Además, se integra con múltiples herramientas de compilación y scripting para automatizar tareas rutinarias.

VS Code es compatible con sistemas de control de versiones como Git, proporcionando un visor de diferencias, historial de cambios y gestión de cambios pendientes. La Figura 1.4 muestra la interfaz principal del editor con un proyecto C abierto.

Para el desarrollo específico en plataformas de 8 bits, VS Code cuenta con una amplia gama de extensiones y plugins que facilitan el trabajo con estas arquitecturas. La Tabla 1.3 detalla algunas de las extensiones más relevantes disponibles para este tipo de desarrollo.

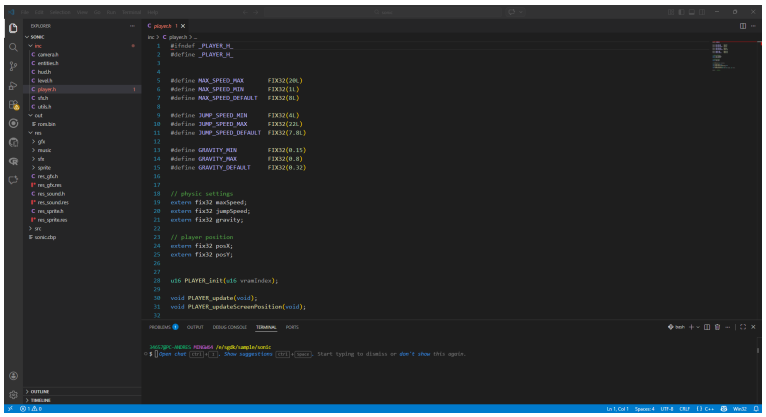

Figura 1.4: Captura de pantalla de Visual Studio Code.

Tabla 1.3: Plugins de VS Code para desarrollo en plataformas de 8 bits

Plugin / Extensión	**Lenguaje / Sistema**	**Función principal**	**Notas**
Retro Assembler	6502, Z80, 6809	Assembler + resaltado de sintaxis	Compatible con NES, C64, ZX Spectrum
vscode-asm6502	6502	Resaltado de sintaxis y snippets	Ideal para NES y Commodore 64
z80-code	Z80	Resaltado de sintaxis y snippets	Compatible con ZX Spectrum, Amstrad CPC
GBDK Support	C (GBDK-2020)	Compilación y snippets para Game Boy	Incluye build tasks y ejemplos
ZX Basic Studio Integration	Boriel Basic	Soporte de sintaxis y proyectos ZX Basic	Requiere configuración manual de paths
C64 Debugger	C64	Depurador integrado para VICE	Funciona con emulador C64
NES Debugger	6502 / NES	Depuración de ROMs	Integración con FCEUX
Arduino / PlatformIO	C / C++	Compilación y depuración hardware	Útil para MCU de 8 bits (AVR, 8051)
GBDK Snippets	C (Game Boy)	Snippets para librerías GBDK	Acelera desarrollo de gráficos y sonido
VIC-20 / C64 Emulation	6502	Integración con emuladores	Ejecuta código en entornos simulados
Retro Debug	Multi Z80 / 6502	Depuración general retro	Soporta snapshots y breakpoints
ZX Spectrum Next Tasks	Z80 / BASIC	Build y ejecución de programas	Compatible con emuladores y hardware real

1.3.5. Eclipse CDT + toolchains

Eclipse es un IDE de código abierto y multiplataforma [11], originalmente diseñado para desarrollo en Java, pero actualmente usado para una amplia variedad de lenguajes y sistemas. Apareció en 2001 como proyecto de IBM y posteriormente pasó a ser administrado por la Eclipse Foundation.

La Figura 1.5 muestra la interfaz de Eclipse CDT con un proyecto C configurado, donde se pueden apreciar los diferentes paneles y herramientas disponibles para el desarrollo.

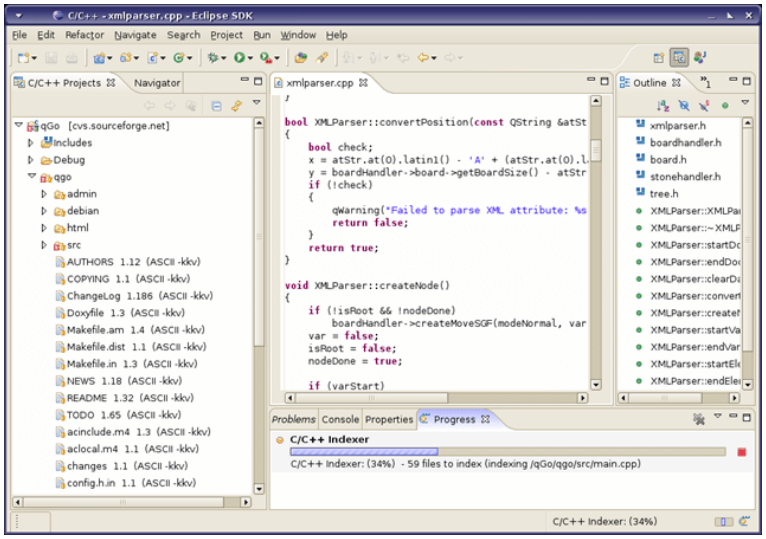

Figura 1.5: Captura de pantalla de Eclipse CDT.

CDT (C/C++ Development Tooling) [12] es un conjunto de extensiones que habilita desarrollo en C y C++, incluyendo compilación, depuración, navegación de código, autocompletado y análisis estático. Eclipse CDT es, por tanto, una versión de Eclipse adaptada para proyectos en C. Por sí solo no incluye toolchains, por lo que debe integrarse con compiladores externos como GCC, SDCC, z88dk o GBDK. Esta modularidad permite su uso en plataformas de 8 bits, aunque requiere configuración manual.

La integración de toolchains externas en Eclipse CDT se realiza mediante la configuración de compiladores y enlazadores dentro del sistema de construcción (Build System). El entorno permite definir rutas a ejecutables, parámetros de compilación, scripts personalizados y perfiles de construcción diferenciados (Debug/Release). En el caso de toolchains como SDCC o z88dk, es necesario definir manualmente los comandos de compilación y enlazado, así como las opciones específicas del microprocesador objetivo.

Entre sus principales ventajas se encuentran la robustez, estabilidad y amplia compatibilidad con toolchains para 8 bits. Sus principales limitaciones son el alto consumo de recursos y la complejidad de configuración inicial.

Eclipse CDT surgió en 2005 como plugin del IDE matriz, permitiendo su aplicación en desarrollo de software embebido y retro gracias a la integración con librerías y herramientas de terceros.

En el contexto específico de desarrollo para sistemas de 8 bits, Eclipse CDT no proporciona asistentes específicos para plataformas retro, por lo que la configuración de entornos como

Game Boy, ZX Spectrum o MSX debe realizarse manualmente. Asimismo, la integración con emuladores requiere scripts externos o configuraciones personalizadas.

La Tabla 1.4 resume las características clave de este entorno de desarrollo para proyectos de 8 bits.

Característica	**Descripción**
IDE completo	Editor, gestión de proyectos y depuración integrada.
Integración toolchains	Compatible con GCC, SDCC, z88dk, GBDK. Múltiples plataformas retro.
Depuración	Soporte GDB y depuradores específicos de MCU/emuladores.
Multiplataforma	Windows, Linux y macOS.
Plugins/Extensible	Soporte para nuevos MCU, lenguajes y emuladores.

Tabla 1.4: Características clave de Eclipse CDT para desarrollo en 8 bits

1.4. Estructura de la memoria

La memoria del proyecto se organiza en varios capítulos que describen de forma progresiva el contexto, desarrollo y resultados del trabajo realizado:

1. *Introducción*: En este apartado se presenta el contexto general del proyecto y la motivación que ha llevado a su desarrollo. Asimismo, se incluye un breve análisis del estado del arte con el fin de situar el trabajo dentro de las soluciones y herramientas existentes.

2. *Análisis*: Se detallan los objetivos del proyecto y las herramientas consideradas para su desarrollo. También se definen los requisitos del sistema y los principales casos de uso que guiarán la implementación.

3. *Diseño*: Se describe el diseño general del sistema, incluyendo la estructura de las pantallas de la aplicación y el flujo de navegación entre ellas.

4. *Implementación*: En este capítulo se explican las funcionalidades desarrolladas y se justifican las principales decisiones técnicas adoptadas durante la implementación.

5. *Pruebas*: Se presenta un conjunto de pruebas destinadas a verificar el correcto funcionamiento del prototipo y a detectar posibles errores en sus funcionalidades principales.

6. *Planificación y presupuesto*: Se incluye la planificación temporal del proyecto mediante un diagrama de Gantt, así como una estimación de los recursos y costes asociados a su desarrollo.

7. *Conclusiones y trabajos futuros*: Finalmente, se recogen las conclusiones derivadas del desarrollo del proyecto, junto con una reflexión sobre los conocimientos adquiridos y posibles líneas de mejora o ampliación para trabajos futuros.

Capítulo 2

Análisis

2.1. Introducción

En esta sección se analizarán los requisitos funcionales del sistema y se definirá el alcance funcional del proyecto.

Para facilitar su desarrollo y comprensión, el trabajo se ha estructurado en varios bloques que pueden abordarse de forma modular:

- Diseño del IDE.
- Diseño del sistema de plugins.
- Desarrollo de un juego de ejemplo.
- Desarrollo de un manual de uso.

El desarrollo de estos bloques se realizará de manera incremental, ya que cada uno de ellos depende en cierta medida de la implementación del anterior. Esta aproximación permite validar progresivamente el funcionamiento del sistema a medida que se incorporan nuevas funcionalidades.

Asimismo, se definirán los distintos casos de uso del sistema, en los que el único actor será el usuario, que será el único que interactúa con la herramienta.

2.2. Objetivos y funciones

El objetivo de este proyecto es el desarrollo de una herramienta multiplataforma que permita la creación de software en lenguaje C para sistemas de 8 bits. Asimismo, la herramienta deberá ser lo suficientemente versátil como para permitir la incorporación de nuevas funcionalidades mediante un sistema de plugins.

El sistema deberá integrar inicialmente las librerías de GBDK-2020 y Z88DK, junto con sus respectivos compiladores basados en el compilador SDCC. Gracias a esta integración, la herramienta podrá compilar software para más de 150 plataformas diferentes [13] [14], la mayoría de ellas basadas en el procesador Z80 de Zilog.

Entre las funciones principales que deberá implementar el sistema se encuentran las siguientes:

- Funcionamiento en sistemas Windows 10.
- Posibilidad de compilar el código fuente en sistemas Linux y macOS.
- Integración con los entornos GBDK-2020 y Z88DK.
- Capacidad para compilar software para múltiples plataformas de 8 bits.
- Visualización de errores y advertencias generados durante la compilación.
- Inclusión de resaltado de sintaxis para el lenguaje C.
- Gestión básica de proyectos y archivos fuente.
- Integración con emuladores para la ejecución del software generado.
- Arquitectura extensible basada en plugins.
- Inclusión de ejemplos de proyectos de referencia.
- Incorporación de un manual de uso para el usuario.

2.3. Requisitos mínimos y restricciones

Para el correcto funcionamiento de la aplicación, se deben cumplir una serie de requisitos mínimos en el equipo en el que se instale:

- Java 17 o superior
- 4 GB de memoria RAM como mínimo (8 GB recomendados)
- 200 MB de espacio disponible en disco
- Compilador de C (como Z88DK, GBDK, entre otros)

En cuanto a las restricciones, no se contemplan limitaciones significativas en su uso. No obstante, al tratarse de una aplicación desarrollada en Java, aunque es multiplataforma, no está diseñada para su ejecución en dispositivos móviles ni tabletas.

2.4. Modelo de desarrollo utilizado

Para el desarrollo de este proyecto se ha seguido un enfoque iterativo o incremental. A partir de un primer prototipo muy básico, se han ido añadiendo nuevas funcionalidades y probándolas de forma progresiva.

En proyectos de cierta complejidad, este enfoque presenta la ventaja de permitir la detección y corrección temprana de errores, antes de su integración en el conjunto principal del sistema.

Este método ha facilitado la adaptación a posibles cambios en los requisitos, así como la mejora continua de la aplicación. No obstante, se ha identificado como principal limitación la falta de una planificación más estructurada, lo que ha derivado en una menor organización en las fases iniciales del proyecto.

2.5. Tecnologías

2.5.1. Introducción

Este proyecto, es fundamentalmente una aplicación desarrollada en Java, lo que condiciona en gran medida las tecnologías empleadas en su desarrollo. Como lenguaje de programación se ha utilizado Java, mientras que para el desarrollo diario se ha empleado el entorno de desarrollo integrado IntelliJ IDEA.

Para la interfaz gráfica se ha optado por el uso de la librería JavaFX, que, aunque el ecosistema Java no destaca especialmente por el desarrollo de interfaces gráficas modernas, permite dotar a la aplicación de un aspecto más actual y visualmente atractivo.

El control de versiones y la gestión del proyecto se han llevado a cabo mediante el uso conjunto de Maven y GitHub.

2.5.2. Java

Java es un lenguaje de programación de propósito general, orientado a objetos y concurrente, diseñado para facilitar el desarrollo de aplicaciones de manera sencilla y segura. Está basado en clases y presenta similitudes con lenguajes como C y C++, aunque simplifica su uso al eliminar ciertas características complejas e incorporar mecanismos que mejoran la seguridad y la portabilidad. Se trata de un lenguaje de tipado estático, en el que los errores pueden detectarse en tiempo de compilación, diferenciándose claramente de los errores en tiempo de ejecución. Además, Java se caracteriza por su independencia de plataforma gracias al uso de la máquina virtual Java (JVM), que permite ejecutar el mismo código en distintos sistemas. Asimismo, incluye gestión automática de memoria mediante un recolector de basura, lo que reduce los problemas asociados a la gestión manual de memoria y contribuye a la robustez del software.[15]

Originalmente, Java fue anunciado en mayo de 1995 por Sun Microsystems, aunque no fue lanzado al mercado hasta el 23 de enero de 1996 [16]. En la actualidad, Java pertenece a la empresa Oracle Corporation, tras la adquisición de Sun Microsystems por parte de esta.

La elección de Java como lenguaje principal del proyecto se fundamenta en varios aspectos clave. En primer lugar, su carácter multiplataforma permite desarrollar aplicaciones que pueden ejecutarse en distintos sistemas operativos sin necesidad de modificaciones, lo que aporta versatilidad y facilita su integración en diferentes entornos de trabajo. En segundo lugar, Java dispone de un amplio ecosistema de herramientas y tecnologías, como JavaFX o Maven, que resultan fundamentales para el desarrollo de interfaces gráficas y la gestión de dependencias del proyecto. Por último, su enfoque basado en la Programación Orientada a Objetos (POO) favorece la escalabilidad y el mantenimiento del software, lo que contribuye a una mejor organización del sistema.

2.5.3. Maven

Maven es una herramienta para la gestión y construcción de proyectos software que va más allá de la simple compilación del código. A través de la gestión del archivo pom.xml (Project Object Model), proporciona un modelo de construcción uniforme y estructurado. Además, se basa en el principio de "convención sobre configuración", lo que simplifica la organización del proyecto y reduce la necesidad de configuraciones explícitas.

Maven incorpora objetivos (goals) predefinidos que permiten ejecutar tareas específicas, como la compilación, las pruebas o el empaquetado del software. En este proyecto, se ha utilizado para la gestión de dependencias y para automatizar las distintas fases del ciclo de construcción, incluyendo la compilación y el empaquetado de la aplicación.

2.5.4. GitHub

GitHub es una plataforma web que ofrece servicios de alojamiento de código fuente y facilita la colaboración entre desarrolladores. Utiliza Git como sistema de control de versiones y permite el trabajo colaborativo en equipo. En la actualidad, GitHub pertenece a Microsoft y dispone de numerosas funcionalidades orientadas a la colaboración, lo que facilita tanto el mantenimiento de proyectos open source como la participación en ellos.

Por su parte, Git es un sistema de control de versiones distribuido desarrollado por Linus Torvalds en el año 2005, y se ha convertido en el estándar de facto en la industria dentro de su ámbito.

En este proyecto, se ha utilizado GitHub como plataforma de alojamiento, estando disponible el repositorio en la siguiente dirección: `https://github.com/ginjol83/samaruc-editor`. Esto no solo garantiza la trazabilidad del desarrollo, sino que también facilita su acceso, revisión y posible colaboración por parte de otros desarrolladores.

2.5.5. JavaFX

JavaFX es una librería estándar de Java, incorporada desde versiones posteriores, que se utiliza para el desarrollo de interfaces gráficas modernas y atractivas. Está basada en componentes (botones, ventanas, layouts, entre otros) y permite la separación entre la lógica y la presentación mediante el uso de archivos FXML.

JavaFX se diseñó como un relevo natural de Java Swing, que había quedado desactualizado en comparación con otras soluciones del mercado. Además de permitir el uso de archivos de presentación basados en XML, incorpora el uso de hojas de estilo CSS para la personalización de la interfaz.

En el desarrollo de este proyecto, JavaFX ha permitido una correcta separación de la capa de presentación dentro del marco del patrón MVC (Model-View-Controller), además de contribuir a la escalabilidad del proyecto y aportar una imagen visual más atractiva.

2.5.6. IntelliJ IDEA

IntelliJ IDEA es un entorno de desarrollo integrado (IDE) desarrollado por JetBrains, orientado principalmente al desarrollo de software en lenguajes como Java y Kotlin, aunque también puede utilizarse como un editor multipropósito para otros lenguajes de programación.

Según su documentación oficial [17], IntelliJ IDEA es un IDE diseñado para el desarrollo profesional, cuyo objetivo es maximizar la productividad del desarrollador. Para ello, incorpora funcionalidades como autocompletado inteligente de código, análisis estático y herramientas avanzadas de refactorización, automatizando tareas rutinarias y repetitivas.

CHAPTER 1 GETTING STARTED

Table 1-1. *IDEA Community vs. Ultimate Editions*

		Ultimate edition	Community edition
Language support	Java	✓	✓
	Groovy	✓	✓
	Kotlin	✓	✓
	Scala (via plugin)	✓	✓
	Python and Jython (via plugin)	✓	✓
	Dart (via plugin)	✓	✓
	Rust (via plugin)	✓	✓
	HTML, XML, JSON, YAML	✓	✓
	XSL, XPath	✓	✓
	Markdown	✓	✓
	JavaScript, TypeScript	✓	
	CoffeeScript, ActionScript	✓	
	SQL	✓	
	CSS, Saas, SCSS, Less, Stylus	✓	
	Ruby and JRuby	✓	
	PHP	✓	
	Go	✓	

(continued)

Figura 2.1: Lenguajes soportados por IntelliJ IDEA según el libro de Hagos (Beginning IntelliJ IDEA)[2].

La elección de este entorno para el desarrollo del presente proyecto se justifica por su amplia adopción en el ámbito profesional del desarrollo en Java, así como por la gran cantidad de documentación disponible, tanto por parte de JetBrains como de la comunidad. Además, ofrece un alto grado de personalización, lo que mejora la experiencia de desarrollo y facilita la adaptación a las necesidades del programador. En este IDE se ha desarrollado íntegramente el código fuente en Java, así como la gestión de dependencias y la construcción del proyecto mediante Maven.

2.5.7. Emulicious

Emulicious es un emulador multiplataforma que permite la ejecución de ROMs de diversas plataformas de 8 bits, aunque su sistema principal objetivo es la Game Boy. Fue desarrollado por Michael Stegmaier en 2013 y, desde entonces, ha recibido actualizaciones periódicas. Además, es un proyecto de código abierto (open source) y cuenta con una comunidad de desarrollo muy activa, con su propio canal en Discord. Está desarrollado íntegramente en Java, lo cual se aprovecha en este proyecto, ya que permite su integración directa en el editor como una biblioteca Java adicional.

Se trata de un emulador con funcionalidades avanzadas orientadas al desarrollo, entre las que destaca la inclusión de un depurador de código ensamblador que permite seguir la evolución de los registros y facilitar la depuración de funcionalidades. Entre sus características se encuentran el resaltado de sintaxis personalizable, el registro de trazas, los puntos de interrupción y los puntos de observación. Además, permite cargar archivos fuente para su visualización directa, evitando así la necesidad de desensamblar la ROM [18].

El hecho de incorporar este conjunto de herramientas específicas para el desarrollo de software hace que se trate de un emulador claramente orientado al desarrollador, en contraposición a otros más enfocados al usuario final, como se puede observar en la siguiente captura.

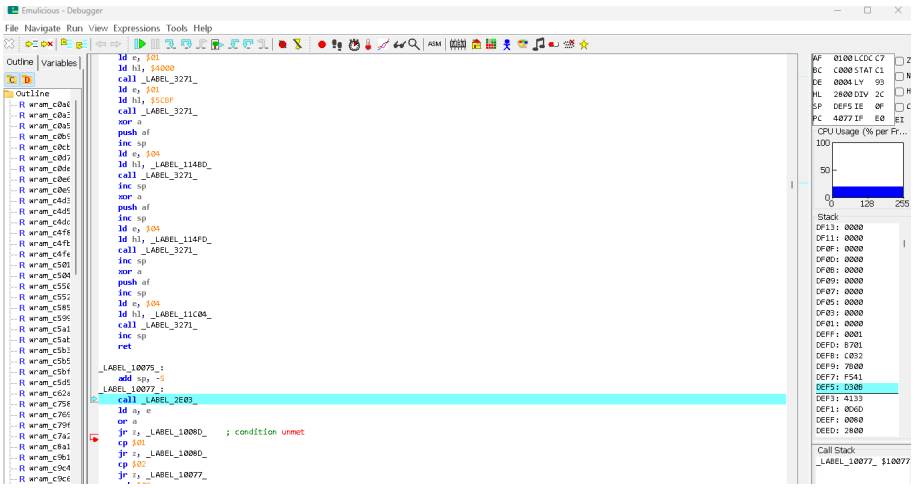

Figura 2.2: Interfaz del debugger integrado de Emulicious mostrando las herramientas de desarrollo disponibles.

El listado de plataformas soportadas es el siguiente:

- Nintendo Game Boy
- Nintendo Game Boy Color
- Sega Master System
- Sega Game Gear
- MSX

2.5.8. JSpeccy

JSpeccy es un emulador de ZX Spectrum desarrollado en Java por José Luis Sánchez [19]. Su objetivo principal es reproducir de forma fiel el comportamiento del hardware original, permitiendo la ejecución de software diseñado para este sistema.

Al igual que otras herramientas utilizadas en este proyecto, JSpeccy está desarrollado íntegramente en Java, lo que facilita su integración dentro del entorno del editor como una biblioteca adicional. Esta característica permite ejecutar y probar directamente los programas desarrollados sin necesidad de depender de aplicaciones externas, simplificando el flujo de trabajo.

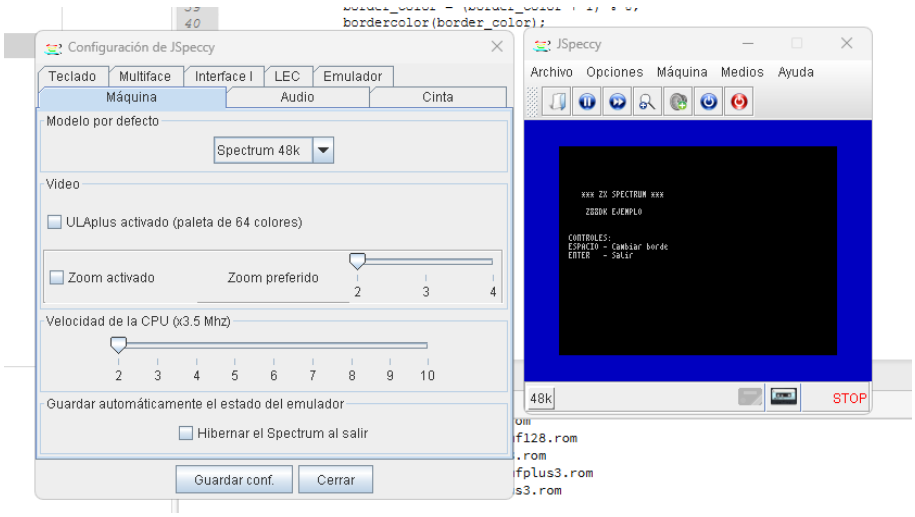

Figura 2.3: Interfaz de JSpeccy y sus diferentes opciones de configuración.

El emulador ofrece soporte para distintos modelos de ZX Spectrum y es compatible con formatos habituales como instantáneas (SNA, Z80) y cintas (TAP, TZX), lo que permite cargar y ejecutar tanto programas como juegos de forma sencilla. Además, incluye soporte para periféricos como el joystick Kempston, ampliando las posibilidades de interacción con el software emulado.

JSpeccy está concebido principalmente como una herramienta de ejecución y validación del software desarrollado, más que como un entorno avanzado de depuración. No obstante, su fidelidad en la emulación y su integración en Java lo convierten en una opción adecuada dentro del flujo de desarrollo planteado en este proyecto.

2.6. Descripción del modelo

En esta sección se describen los requisitos del sistema, entendidos como el conjunto de funcionalidades y restricciones que debe cumplir el entorno de desarrollo propuesto.

Dichos requisitos se presentan desde un punto de vista externo, centrado en el usuario, sin entrar en detalles de implementación interna.

Se distinguen dos tipos de requisitos:

- Requisitos funcionales (RF), que describen las capacidades y servicios que ofrece el sistema.
- Requisitos no funcionales (RNF), que definen atributos de calidad y restricciones del sistema, como el rendimiento, la usabilidad o la portabilidad.

2.6.1. Requisitos

A continuación, se presenta el listado de requisitos identificados para el sistema.

2.6.1.1. Listado de requisitos

Requisitos funcionales

- RF-01: El sistema deberá permitir la creación de nuevos proyectos.
- RF-02: El sistema deberá permitir abrir proyectos existentes.
- RF-03: El sistema deberá permitir guardar proyectos.
- RF-04: El sistema deberá permitir la creación, edición y eliminación de archivos de código.
- RF-05: El sistema deberá permitir compilar proyectos utilizando diferentes toolchains (GBDK, z88dk).
- RF-06: El sistema deberá mostrar los errores de compilación al usuario.
- RF-07: El sistema deberá permitir ejecutar el programa compilado en un emulador.
- RF-08: El sistema deberá permitir la configuración de rutas de compiladores y emuladores.
- RF-09: El sistema deberá soportar la instalación de plugins.
- RF-10: El sistema deberá permitir la gestión (activar/desactivar) de plugins.

Requisitos no funcionales

- RNF-01: El sistema deberá ser multiplataforma (Windows principalmente).
- RNF-02: La interfaz deberá ser intuitiva y fácil de usar.
- RNF-03: El sistema deberá responder a las acciones del usuario en menos de 1 segundo en condiciones normales.
- RNF-04: El sistema deberá permitir la extensión mediante plugins sin modificar el núcleo.
- RNF-05: El sistema deberá estar desarrollado en Java.

2.6.2. Casos de uso

Los casos de uso describen las interacciones entre los actores y el sistema, representando de forma clara cómo se utilizan las funcionalidades definidas en los requisitos. Permiten modelar el comportamiento del sistema desde el punto de vista del usuario, mostrando los distintos escenarios de uso.

En este proyecto, el actor principal es el usuario desarrollador, que utiliza el entorno para crear, editar, compilar y ejecutar programas para sistemas de 8 bits.

A continuación, se describen los principales casos de uso identificados:

- **CU-01: Crear proyecto** - El usuario crea un nuevo proyecto especificando su nombre y ubicación en el sistema de archivos.
- **CU-02: Abrir proyecto** - El usuario selecciona un proyecto existente para cargarlo en el entorno de desarrollo.
- **CU-03: Guardar proyecto** - El usuario guarda los cambios realizados en el proyecto actual.
- **CU-04: Gestionar archivos de código** - El usuario puede crear, editar y eliminar archivos de código fuente dentro del proyecto.
- **CU-05: Compilar proyecto** - El usuario ejecuta el proceso de compilación utilizando una toolchain seleccionada (GBDK o z88dk).
- **CU-06: Visualizar errores de compilación** - El sistema muestra los errores generados durante la compilación, permitiendo al usuario corregirlos.
- **CU-07: Ejecutar programa en emulador** - El usuario ejecuta el programa compilado en un emulador integrado o externo.
- **CU-08: Configurar entorno** - El usuario define las rutas de compiladores y emuladores necesarios para el funcionamiento del sistema.
- **CU-09: Gestionar plugins** - El usuario instala, activa o desactiva plugins para extender la funcionalidad del entorno.

2.6.2.1. Diagrama de casos de uso

Los diagramas de casos de uso permiten representar de manera visual las interacciones entre los actores y el sistema, facilitando la comprensión de los distintos escenarios de uso. A través de estos diagramas, es posible observar de un vistazo qué funcionalidades ofrece el sistema y cómo se relacionan con los usuarios.

En este proyecto, el actor principal es el usuario desarrollador, que realiza acciones sobre el entorno de desarrollo para crear, gestionar y ejecutar proyectos de software.

El diagrama principal de casos de uso incluye los siguientes elementos:

- **Actor:** Usuario desarrollador
- **Casos de uso principales:**
  - Crear proyecto
  - Abrir proyecto
  - Guardar proyecto
  - Gestionar archivos de código
  - Compilar proyecto
  - Visualizar errores de compilación
  - Ejecutar programa en emulador
  - Configurar entorno
  - Gestionar plugins

A continuación se presenta el diagrama UML correspondiente, donde se representan los casos de uso y su relación con el actor:

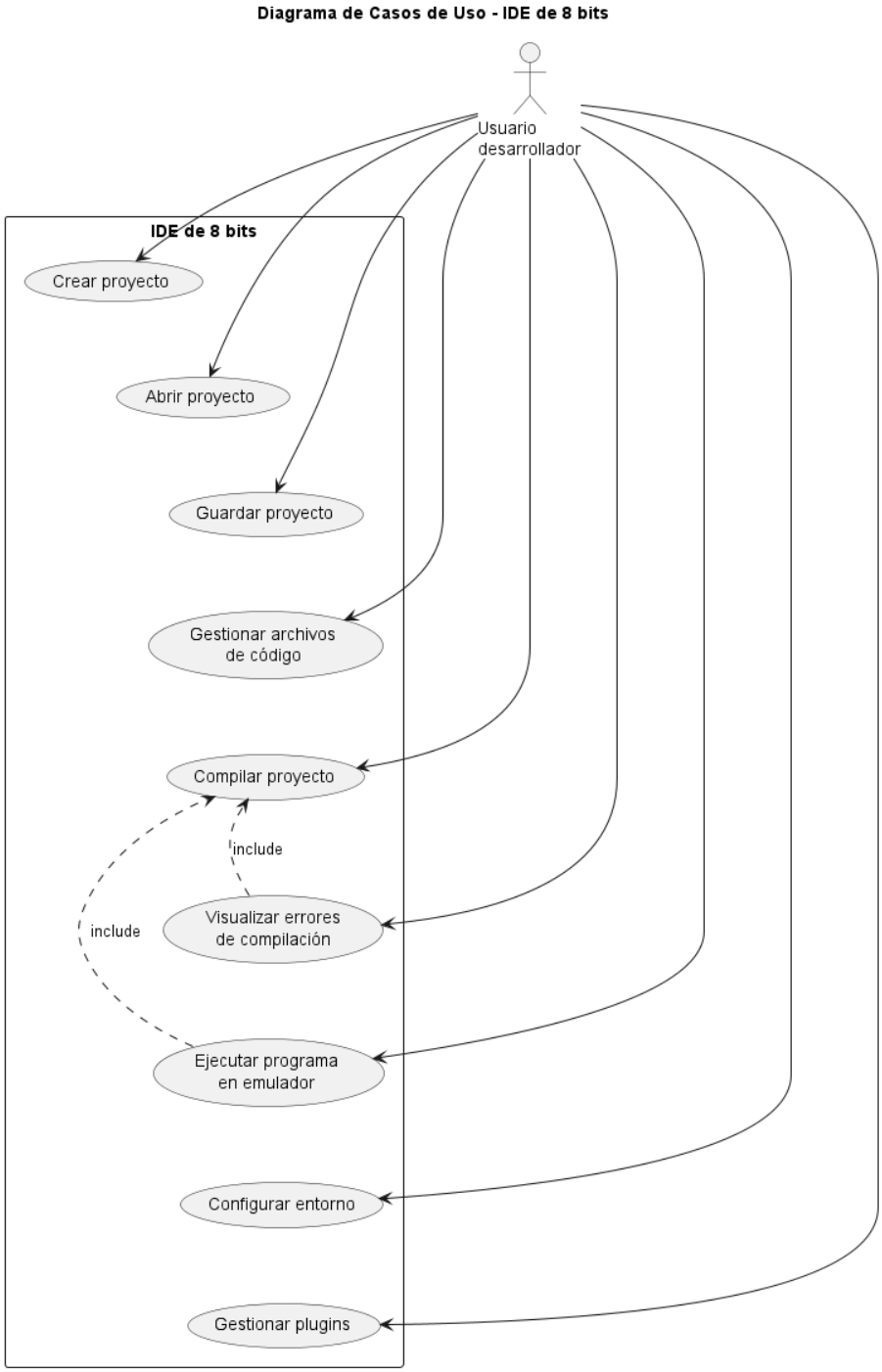

Figura 2.4: Diagrama de casos de uso del sistema de desarrollo IDE.

2.6.3. Casos de uso

Los casos de uso permiten modelar las interacciones entre los actores y el sistema, mostrando cómo se utilizan las funcionalidades definidas en los requisitos. Cada caso de uso describe un escenario específico en el que el usuario interactúa con el sistema, desde el inicio hasta el resultado esperado.

El actor principal es el **usuario desarrollador**, que utiliza el entorno para crear, editar, compilar y ejecutar programas para sistemas de 8 bits. A continuación se describen los principales casos de uso identificados:

CU-01: Crear proyecto

- **Actor:** Usuario desarrollador
- **Precondición:** Ningún proyecto abierto o el usuario desea iniciar un nuevo proyecto
- **Flujo principal:**
  1. El usuario selecciona la opción "Nuevo proyecto".
  2. El sistema solicita nombre y ubicación del proyecto.
  3. El usuario introduce los datos y confirma.
  4. El sistema crea el proyecto y lo abre en el entorno.
- **Postcondición:** Proyecto creado y listo para añadir archivos de código.

CU-02: Abrir proyecto

- **Actor:** Usuario desarrollador
- **Precondición:** Existen proyectos previamente guardados
- **Flujo principal:**
  1. El usuario selecciona "Abrir proyecto".
  2. El sistema muestra un selector de proyectos disponibles.
  3. El usuario selecciona un proyecto y confirma.
  4. El sistema carga el proyecto en el entorno.
- **Postcondición:** Proyecto cargado y listo para edición o compilación.

CU-03: Guardar proyecto

- **Actor:** Usuario desarrollador
- **Flujo principal:**
  1. El usuario selecciona "Guardar proyecto".
  2. El sistema guarda todos los archivos y configuraciones del proyecto.
- **Postcondición:** Cambios del proyecto guardados de manera segura.

CU-04: Gestionar archivos de código

- **Actor:** Usuario desarrollador
- **Flujo principal:**
  1. El usuario crea, edita o elimina archivos de código dentro del proyecto.
  2. El sistema refleja los cambios en la estructura del proyecto.
- **Postcondición:** Archivos actualizados correctamente.

CU-05: Compilar proyecto

- **Actor:** Usuario desarrollador
- **Precondición:** El proyecto contiene archivos de código válidos
- **Flujo principal:**
  1. El usuario selecciona la opción "Compilar".
  2. El sistema utiliza la toolchain configurada para generar el ejecutable.
  3. El sistema muestra los resultados de la compilación (errores o éxito).
- **Postcondición:** Archivo ejecutable generado si no existen errores.

CU-06: Visualizar errores de compilación

- **Actor:** Usuario desarrollador
- **Flujo principal:**
  1. Tras la compilación, el sistema muestra los errores encontrados.
  2. El usuario analiza los errores y realiza correcciones en el código.
- **Postcondición:** El usuario dispone de la información necesaria para corregir errores.

CU-07: Ejecutar programa en emulador

- **Actor:** Usuario desarrollador
- **Flujo principal:**
  1. El usuario selecciona "Ejecutar en emulador".
  2. El sistema ejecuta el programa compilado en Emulicious o JSpeccy según la configuración.
- **Postcondición:** Programa ejecutado y comportamiento observable por el usuario.

CU-08: Configurar entorno

- **Actor:** Usuario desarrollador
- **Flujo principal:**
  1. El usuario accede a la configuración del sistema.
  2. Define rutas de compiladores y emuladores.
- **Postcondición:** Sistema preparado para compilar y ejecutar programas correctamente.

CU-09: Gestionar plugins

- **Actor:** Usuario desarrollador
- **Flujo principal:**
  1. El usuario instala, activa o desactiva plugins.
  2. El sistema aplica los cambios de manera inmediata.
- **Postcondición:** Plugins activos o desactivados según la decisión del usuario.

2.6.4. Diagrama de estados

El diagrama de estados describe los distintos estados por los que puede pasar el sistema durante su ejecución, así como las transiciones entre dichos estados en función de las acciones del usuario o de eventos internos.

En este caso, se modela el comportamiento del entorno de desarrollo en relación con la gestión de proyectos, desde su creación o apertura hasta su compilación y ejecución.

El sistema comienza en un estado inicial en el que no hay ningún proyecto cargado. A partir de ahí, el usuario puede crear o abrir un proyecto, pasando al estado de proyecto cargado. Desde este estado, el usuario puede editar el código, compilar el proyecto o cerrar el mismo.

Durante el proceso de compilación, el sistema puede transitar a un estado de error de compilación si se detectan fallos, o a un estado de compilación correcta en caso de éxito. En este último caso, el usuario puede ejecutar el programa en un emulador.

Finalmente, el sistema puede volver a estados anteriores en función de las acciones del usuario, como la corrección de errores o el cierre del proyecto.

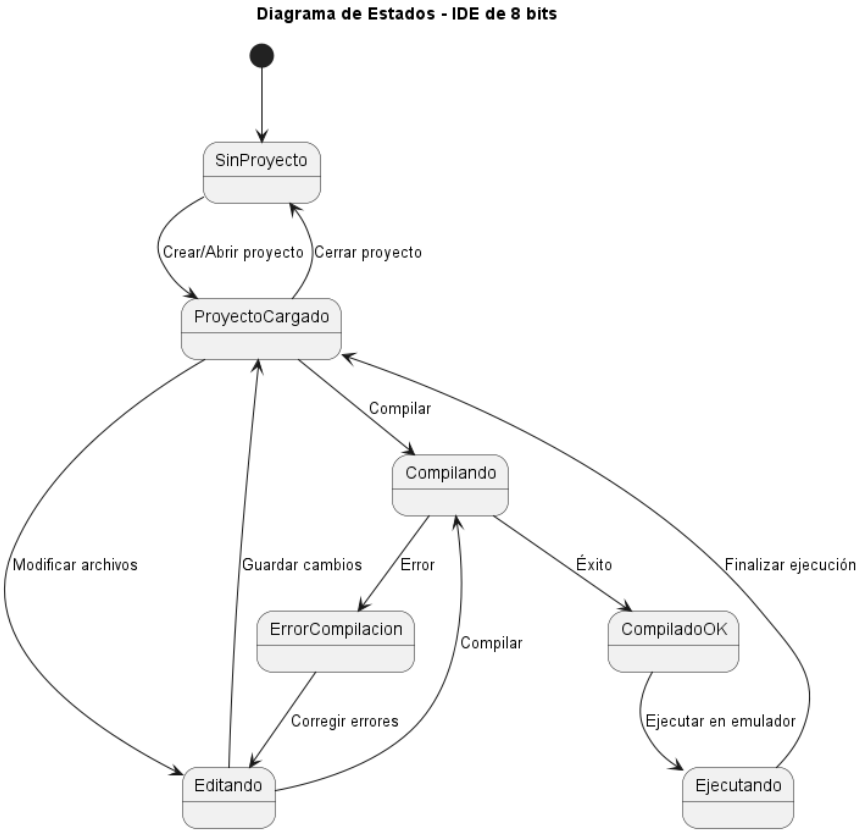

Figura 2.5: Diagrama de estados del entorno de desarrollo.

2.6.5. Diagrama de clases

El diagrama de clases representa la estructura estática del sistema, mostrando las clases principales, sus atributos, métodos y las relaciones entre ellas.

En este proyecto, se ha diseñado un modelo orientado a objetos que refleja los distintos componentes del entorno de desarrollo. La clase principal es el IDE, que actúa como núcleo del sistema y coordina las operaciones principales, como la gestión de proyectos, la compilación y la ejecución.

El sistema se organiza en torno a la clase Proyecto, que contiene los archivos de código fuente. Para la compilación y ejecución, se utilizan las clases Compilador y Emulador, respectivamente. Además, se incluye un sistema de extensión mediante plugins, gestionado por la clase GestorPlugins.

Por último, la clase Configuracion permite almacenar los parámetros necesarios para el funcionamiento del sistema, como las rutas de compiladores y emuladores.

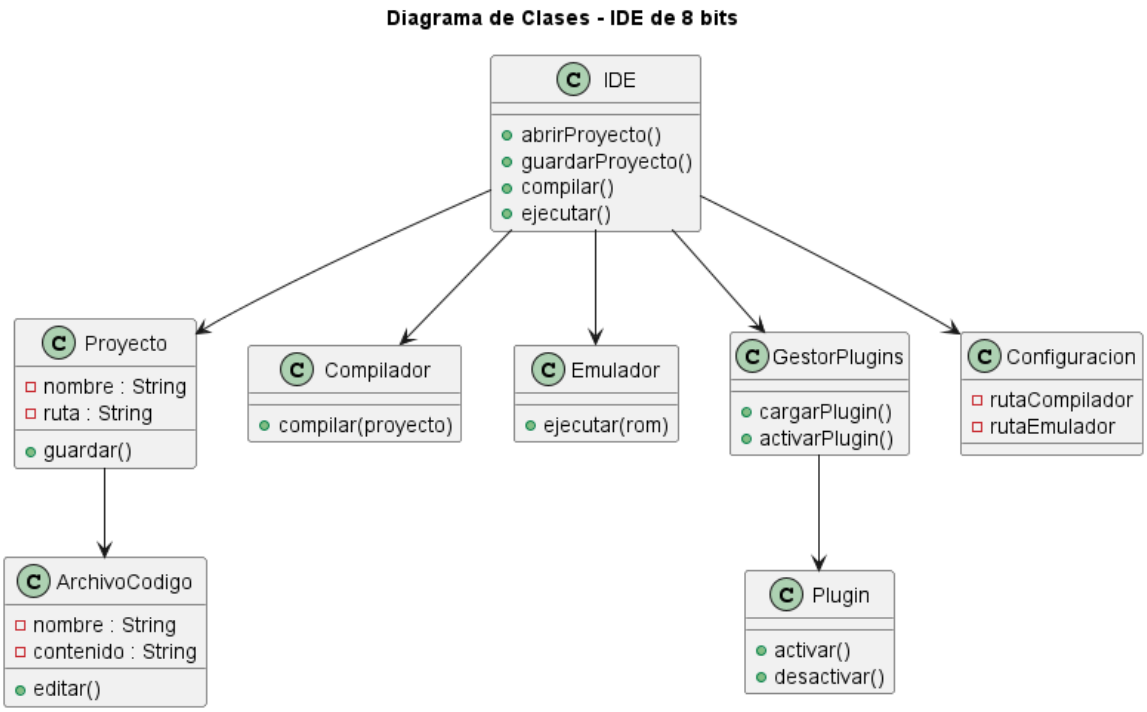

Figura 2.6: Diagrama de clases del sistema.

Capítulo 3

Diseño

3.1. Diseño del sistema

3.1.1. Arquitectura

3.1.1.1. Modelo

3.1.1.2. Vista

3.1.1.3. Controlador

3.1.2. Diseño de vistas

3.1.2.1. Configuración

La pantalla de configuración es una de las pantallas más importantes de la aplicación, ya que desde esta pantalla controlamos todos los aspectos personalizables de la aplicación, como el aspecto, idioma o incluso los parámetros para el compilador.

Con el motivo de la multifuncionalidad de esta vista, se ha diseñado con un sistema de pestañas y sub-pestañas, en el que cada pestaña es un aspecto de la aplicación, básicamente lo dividiríamos en tres pestañas principales diferenciadas: idioma, compilador y plugins.

3.1.2.1.1 Configuración de idioma

La configuración de idioma permite al usuario seleccionar el idioma de la interfaz de la aplicación, ofreciendo soporte multiidioma para una mejor experiencia de usuario.

3.1.2.1.2 Configuración de compiladores

El sistema permite configurar diferentes toolchains de compilación según el target de desarrollo. Se incluyen configuraciones específicas para GBDK-2020 y Z88DK.

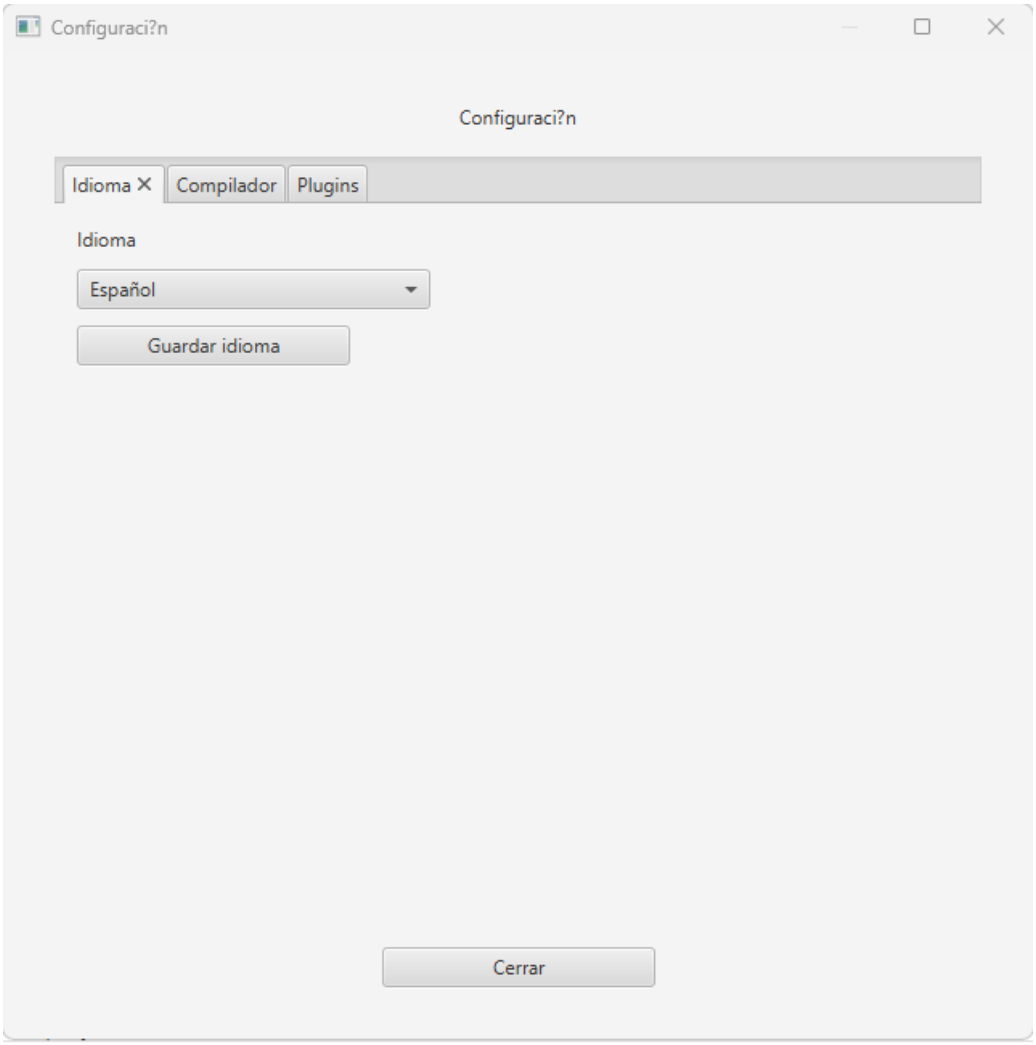

Figura 3.1: Diálogo de configuración de idioma.

3.1.2.1.3 Configuración de plugins

La gestión de plugins permite instalar, activar, desactivar y configurar extensiones del sistema, proporcionando flexibilidad y extensibilidad a la aplicación.

3.1.2.2. Diálogos auxiliares

El sistema incluye diversos diálogos auxiliares que facilitan la interacción del usuario con la aplicación.

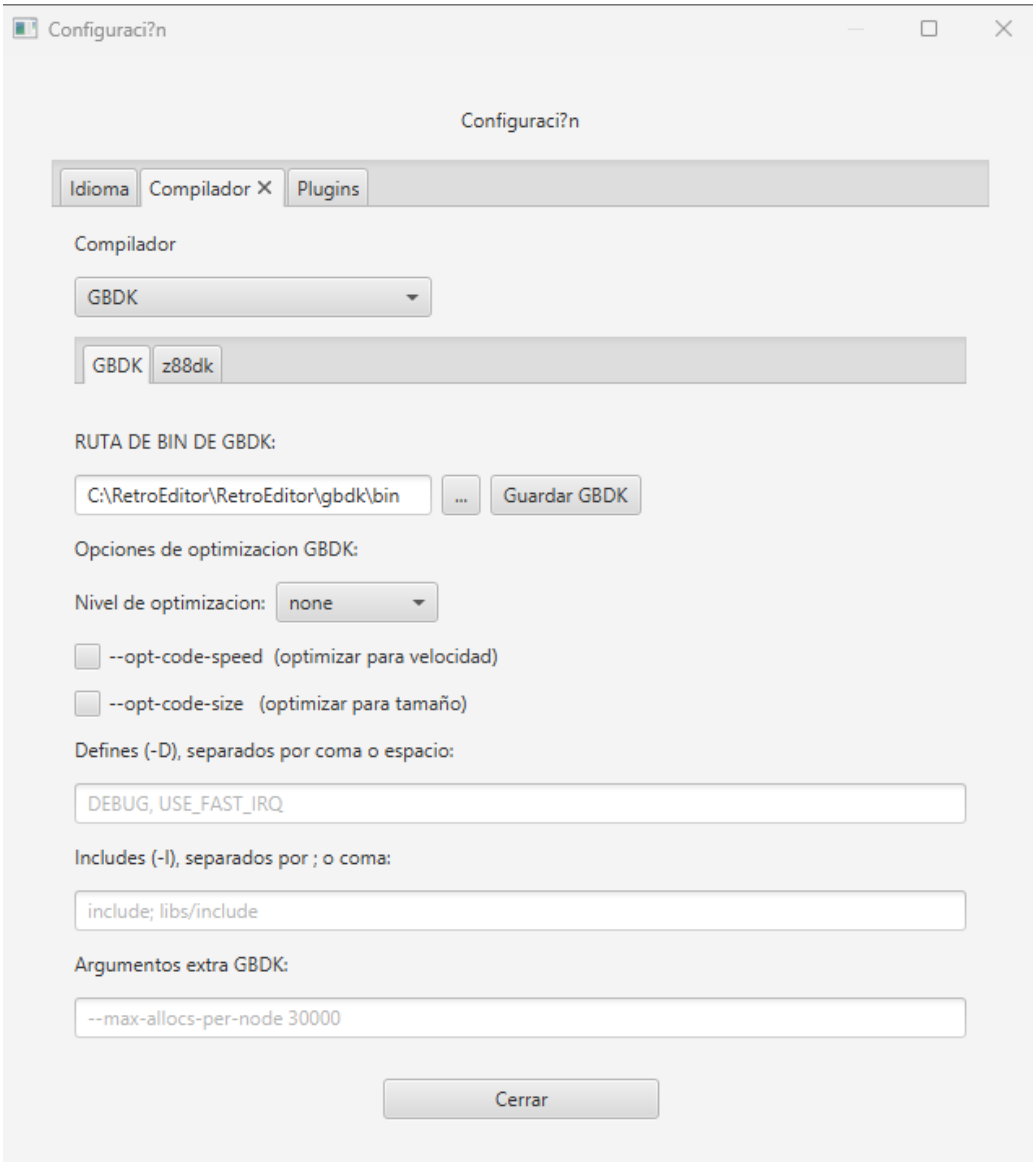

Figura 3.2: Diálogo de configuración del compilador GBDK-2020.

3.1.2.2.1 Creación de nuevos proyectos

3.1.2.2.2 Funcionalidades de búsqueda

El sistema proporciona herramientas de búsqueda tanto en archivos individuales como a nivel de proyecto.

3.1.2.2.3 Navegación de código

3.1.2.3. Configuración de emuladores

La aplicación integra emuladores para probar el software desarrollado. La configuración de JSpeccy permite ajustar los parámetros de emulación del ZX Spectrum.

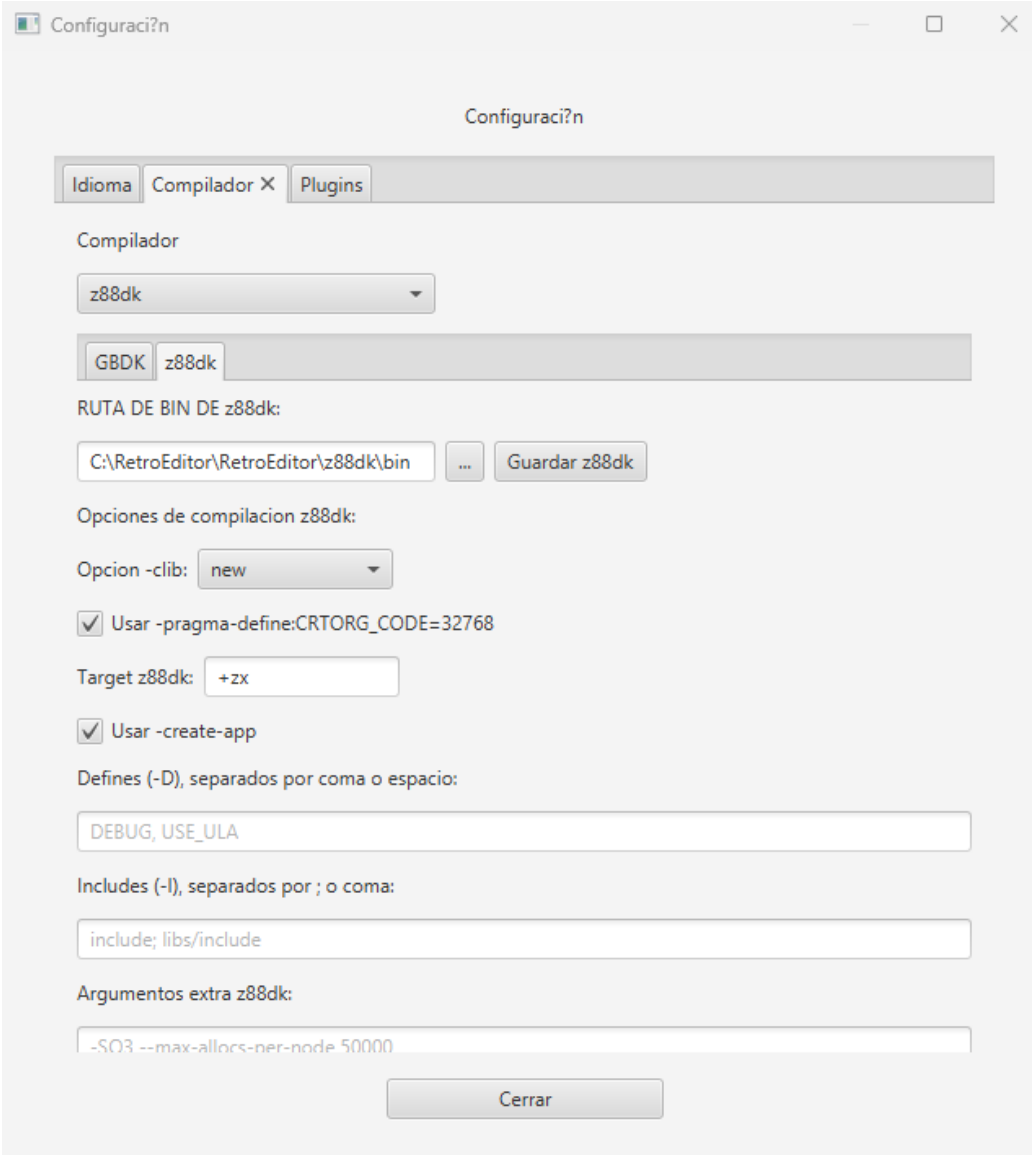

Figura 3.3: Diálogo de configuración del compilador Z88DK.

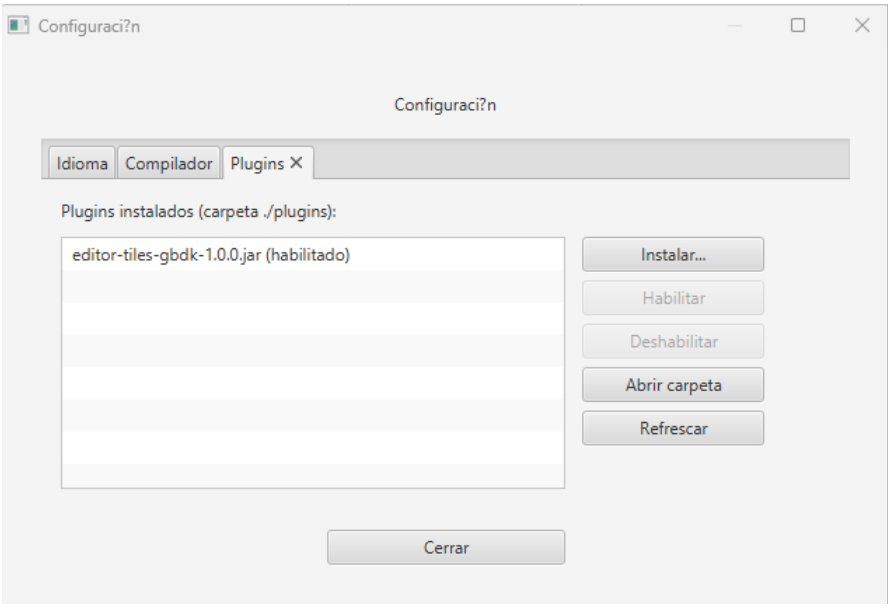

Figura 3.4: Diálogo de configuración de plugins.

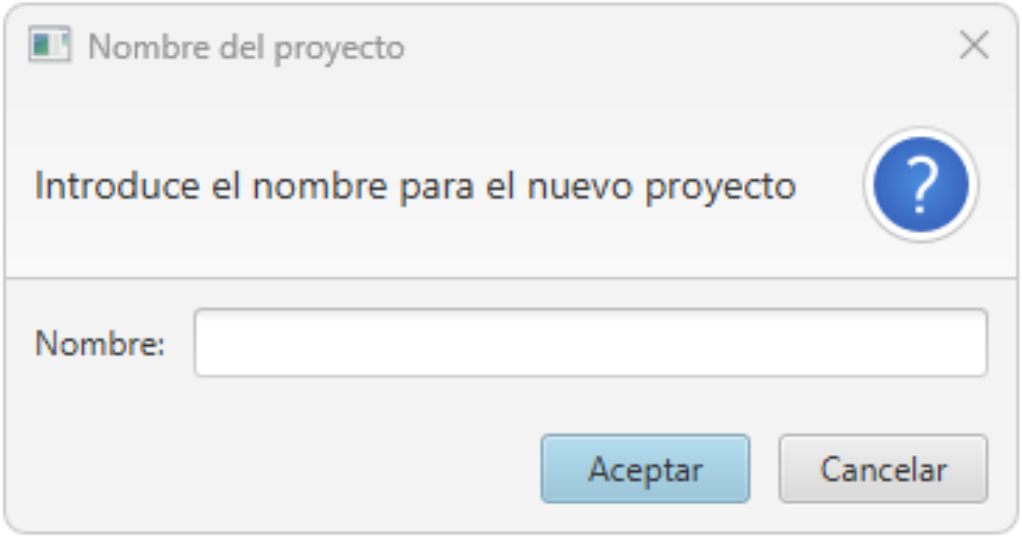

Figura 3.5: Diálogo para la creación de nuevos proyectos.

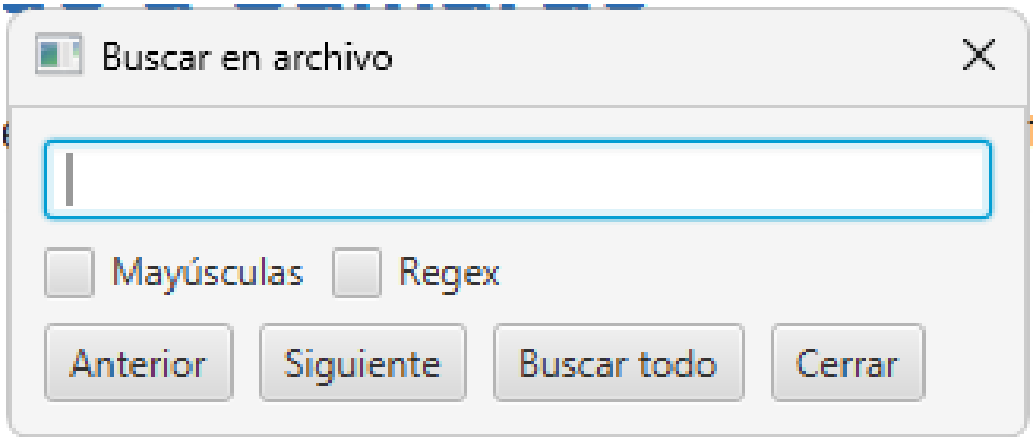

Figura 3.6: Diálogo de búsqueda en archivos.

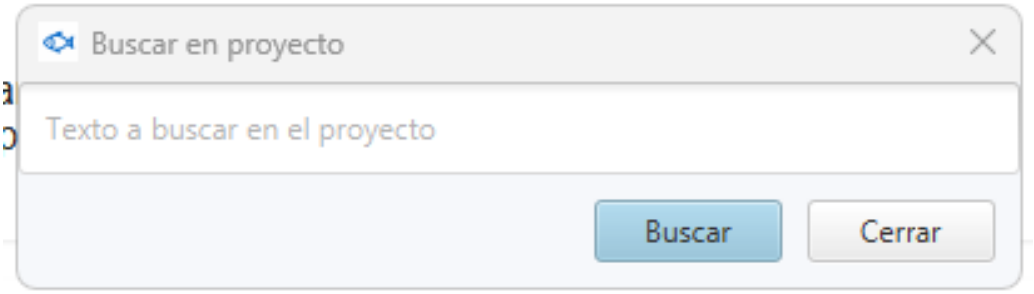

Figura 3.7: Diálogo de búsqueda a nivel de proyecto.

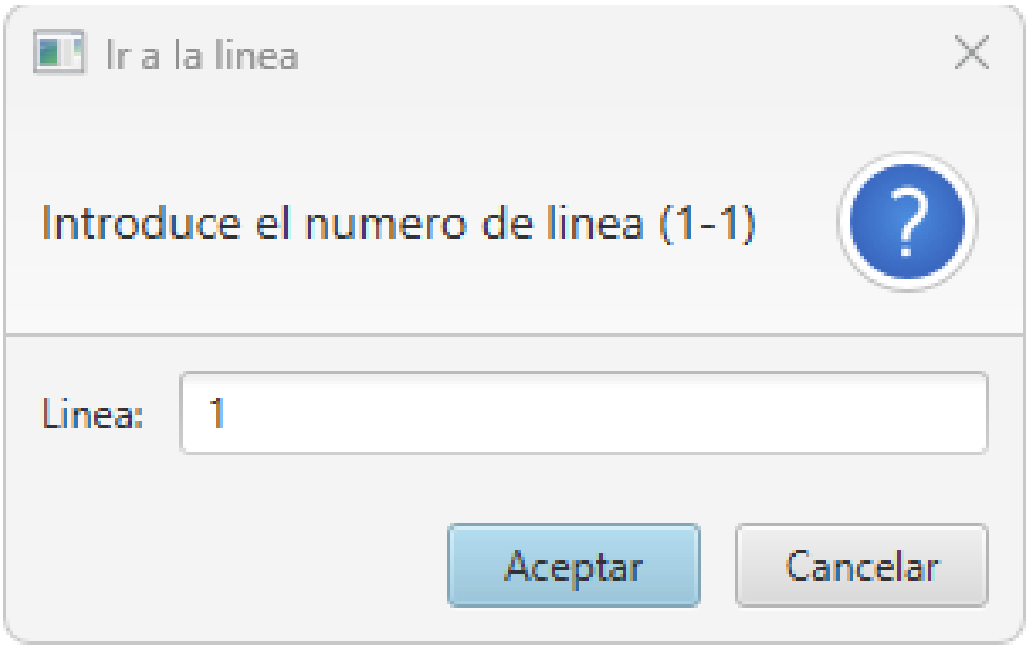

Figura 3.8: Diálogo para ir a una línea específica.

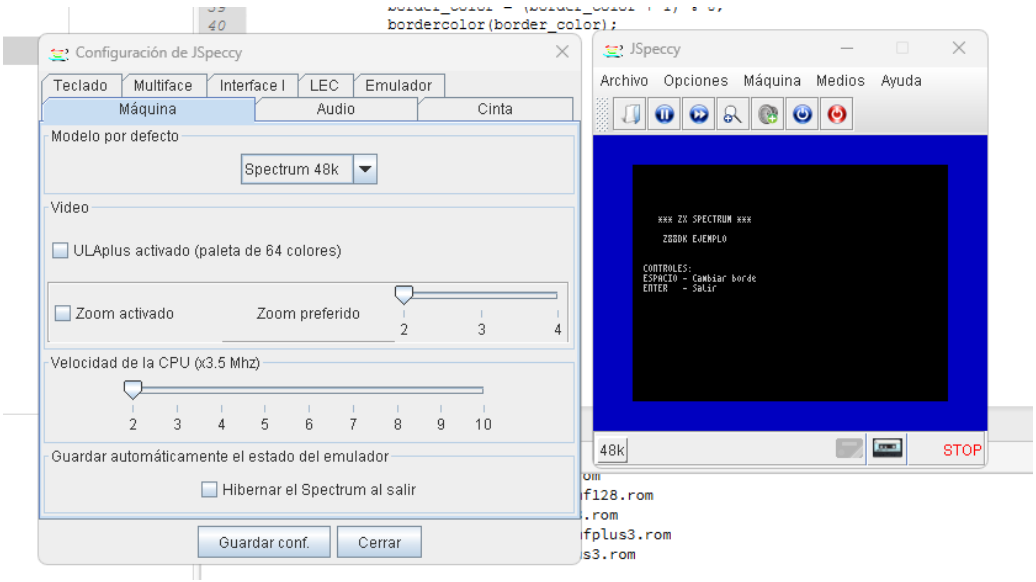

Figura 3.9: Interface de configuración del emulador JSpeccy.

Capítulo 4

Desarrollo del proyecto

Contenido del capítulo...

Capítulo 5

Pruebas

Contenido del capítulo...

Capítulo 6

Conclusiones

Contenido del capítulo...

Bibliografía

[1] z88dk Project. (2024) Supported platforms. Consultado: 6 marzo 2026. [Online]. Available: https://github.com/z88dk/z88dk/wiki/Platform

[2] T. Hagos, *Beginning IntelliJ IDEA: Integrated Development Environment for Java Programming*. Berkeley, CA: Apress, 2022.

[3] GBDK Team. (2020) Gbdk-2020: Gbdk is a cross-platform development kit for sm83, z80 and 6502 based gaming consoles. it includes libraries, toolchain utilities and the sdcc c compiler suite. Acceso: 2023-10-23. [Online]. Available: https://gbdk.org/

[4] Z88DK Team. (2023) Z88dk - the z80 development kit. Z88DK Project. Acceso: 2026-03-03. [Online]. Available: https://www.z88dk.org/

[5] Orange FM Team. (2023) Orange fm - modern cartridge for game boy family functioning as a stereo radio system. Orange FM Project. Acceso: 2026-03-03. [Online]. Available: https://orangegb.com/orangefm/

[6] marco's retrobits. (2025) Znake (zx spectrum). Acceso: 2026-03-06. [Online]. Available: https://retrobits.itch.io/znake

[7] ZX Basic Studio Team. (2024) Zx basic studio - a modern development environment for zx spectrum. Acceso: 2026-03-06. [Online]. Available: https://github.com/boriel-basic/ZXBasicStudio

[8] Boriel. (2024) Boriel's basic - a modern basic compiler for z80-based systems. Acceso: 2026-03-06. [Online]. Available: https://github.com/boriel-basic

[9] J. S. D. ("Duefectu"). (2022) Zx graphics - enhanced graphics editor for zx spectrum. Acceso: 2026-03-06. [Online]. Available: https://github.com/Duefectu/ZXGraphics

[10] Visual Studio Code documentation. (2026) Why did we build visual studio code? Acceso: 2026-03-06. [Online]. Available: https://code.visualstudio.com/Docs/editor/whyvscode

[11] Eclipse Foundation. (2026) Eclipse ide. Acceso: 2026-03-06. [Online]. Available: https://www.eclipse.org/

[12] Eclipse CDT Team. (2026) Eclipse cdt - c/c++ development tooling. Acceso: 2026-03-06. [Online]. Available: https://www.eclipse.org/cdt/

[13] Z88DK Team. (2023) Z88dk supported platforms. Z88DK Project. Acceso: 2026-03-03. [Online]. Available: https://www.z88dk.org/wiki/doku.php?id=libnew:platforms

[14] GBDK Team. (2020) Gbdk supported consoles. GBDK Project. Acceso: 2026-03-03. [Online]. Available: https://gbdk.org/docs/api/docs_supported_consoles.html

[15] J. Gosling, B. Joy, G. L. Steele, and G. Bracha, *The Java Language Specification*, java se 8 edition ed. Addison-Wesley, 2014.

[16] Sun Microsystems. (1995) Java announcement. Acceso: 2026-03-06. [Online]. Available: https://web.archive.org/web/20070310235103/http://www.sun.com/smi/Press/sunflash/1996-01/sunflash.960123.10561.xml

[17] JetBrains. (2026) Intellij idea documentation. Acceso: 2026-03-06. [Online]. Available: https://www.jetbrains.com/help/idea/discover-intellij-idea.html

[18] M. Stegmaier. (2013) Emulicious - emulator and debugger for 8-bit systems. Acceso: 2026-04-03. [Online]. Available: https://emulicious.net/

[19] J. L. Sánchez. (2024) Jspeccy - zx spectrum emulator. Acceso: 2026-04-03. [Online]. Available: https://jspeccy.speccy.org/

[20] J. S. Duran, *Boriel Basic para ZX Spectrum.* Torraza, Italia: Amazon autopublishing, 2023.

Apéndice A

Manual de usuario

A.1. Introducción

SamaruC Retro Editor es un entorno de desarrollo integrado (IDE) especializado para programación en lenguaje C, diseñado con una interfaz moderna y funcional basada en JavaFX.

Nota: El nombre *SamaruC* es un juego de palabras. Por una parte *Samaruc* es un pez endémico de la Comunidad Valenciana en grave peligro de extinción, referenciando la región de origen del proyecto. Por otra parte, remarca la terminación con letra mayuscula de la `C`, haciendo referencia al lenguaje de programación para el cual está optimizado este editor.

A.1.1. Características principales

- **Editor de código** con resaltado de sintaxis para C/C++
- **Gestión de proyectos** con explorador de archivos integrado
- **Compilación y ejecución** directa desde el IDE
- **Terminal integrada** con consola de salida y depuración
- **Búsqueda avanzada** en archivos y proyectos completos
- **Pestañas múltiples** para trabajar con varios archivos simultáneamente
- **Interfaz personalizable** con temas claro y oscuro
- **Soporte multiidioma** (Español/Inglés)
- **Deshacer/Rehacer** con historial ilimitado
- **Iconos FontAwesome 5** para una interfaz moderna

A.2. Instalación

A.2.1. Requisitos del Sistema

- **Java:** JDK 17 o superior
- **Maven:** 3.6 o superior (para compilar desde código fuente)
- **Compilador C:** GCC, Z88DK o GBDK (según tu proyecto)
- **Sistema Operativo:** Windows, Linux o macOS

A.2.2. Instalación desde Código Fuente

Para compilar y ejecutar el editor desde el código fuente:

1. Clona o descarga el repositorio del proyecto
2. Abre una terminal en el directorio del proyecto
3. Ejecuta los siguientes comandos:

```
mvn clean compile
mvn install
mvn javafx:run
```

A.2.3. Instalación desde Ejecutable

Si tienes el paquete ejecutable (.exe en Windows):

- Extrae el archivo `RetroEditor-win.zip`
- Ejecuta `RetroEditor.exe`
- No requiere instalación de Java (incluye runtime embebido)

A.3. Interfaz

A.3.1. Descripción de la Interfaz

A.3.1.1. Barra de Menús

La barra de menús se encuentra en la parte superior de la ventana como se puede apreciar en la captura y contiene los siguientes menús principales:

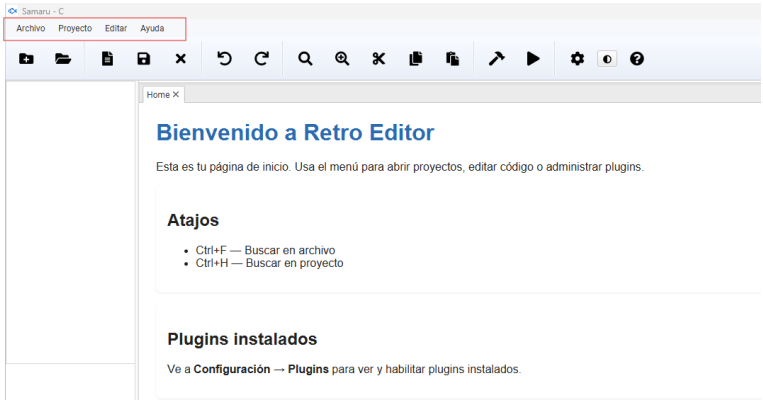

Figura A.1: Situación de los menus desplegables de la herramienta.

Funciones principales de cada menú:

- **Archivo:** Crear, abrir, guardar y cerrar archivos.
- **Edición:** Deshacer, rehacer, cortar, copiar, pegar.
- **Compilación:** Compilar y ejecutar el proyecto actual.
- **Configuración:** Ajustes de idioma, compilador y preferencias.
- **Ayuda:** Acceso a este manual y acerca de.

A.3.1.2. Barra de Herramientas

Como se puede ver en la siguiente captura de pantalla otra manera de acceder rápido a las funciones elementales de la herramienta es mediante la barra de herramientas con iconos representativos de cada función:

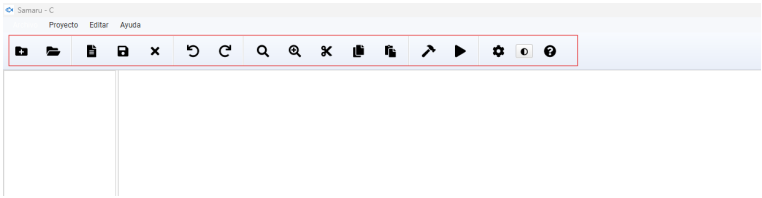

Figura A.2: Barra de herramientas con los iconos representativos de cada función.

La barra de herramientas proporciona acceso rápido a las funciones más utilizadas:

- **Nuevo Proyecto:** Abre el dialogo de creación de un nuevo proyecto
- **Abrir Proyecto:** Abre un proyecto o archivo existente
- **Nuevo:** Crea un archivo nuevo
- **Guardar:** Guardar el archivo actual
- **Cerrar:** Cierra el archivo actual
- **Deshacer:** Desacer la última acción realizada
- **Rehacer:** Rehacer la última acción deshecha
- **Buscar en archivo:** Herramientas de búsqueda en el archivo actual
- **Buscar en proyecto:** Herramientas de búsqueda en todo el proyecto
- **Cortar:** Función para cortar un texto seleccionado
- **Copiar:** Función para copiar un texto seleccionado
- **Pegar:** Función de pegado de texto copiado o cortado
- **Compilar:** Construcción y compilación del código fuente
- **Ejecutar:** Construcción, compilación y ejecución del código fuente
- **Configuración:** Acceso a preferencias y configuración
- **Tema:** Alternar entre tema claro y oscuro
- **Ayuda - Manual:** Abre este manual de usuario

A.3.1.3. Panel del Explorador de Proyectos

Ubicado en el lado izquierdo de la interfaz, como se aprecia en la imagen, este panel muestra la estructura de directorios y archivos del proyecto activo.

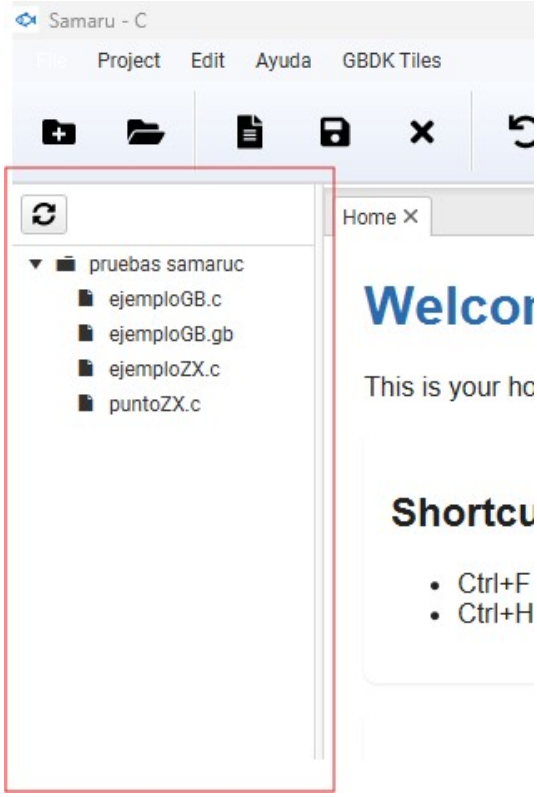

Figura A.3: Panel del explorador de proyectos.

Desde este apartado es posible navegar por la jerarquía de carpetas, así como desplegar o contraer los directorios para visualizar u ocultar su contenido. Asimismo, los archivos pueden abrirse mediante doble clic, lo que permite visualizar y editar su contenido en el área de trabajo principal.

Al hacer clic con el botón derecho del ratón sobre esta zona, se despliega un menú contextual que ofrece opciones como la creación de un nuevo archivo o una nueva carpeta.

Si pulsamos la tecla `Suprimir` con un archivo o carpeta seleccionada, aparecerá un diálogo de confirmación que si confirmamos eliminará el elemento seleccionado.

Además, tenemos un botón en la esquina superior izquierda del panel del explorador que permite refrescar la vista para mostrar los cambios realizados en el sistema de archivos, como la adición o eliminación de archivos fuera del editor.

A.3.1.4. Área de Edición con Pestañas

El área central donde se editan los archivos, como se puede ver en la captura. Características:

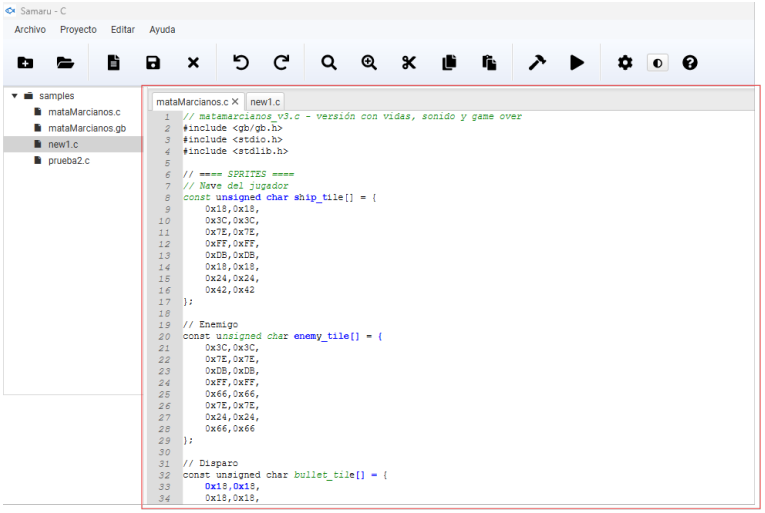

Figura A.4: Área de edición con pestañas.

- Soporte para múltiples archivos abiertos simultáneamente
- Resaltado de sintaxis automático para C/C++
- Numeración de líneas
- Indicador de archivo modificado (*) en la pestaña

A.3.1.5. Panel de Consola Y Monitor + Barra de Estado

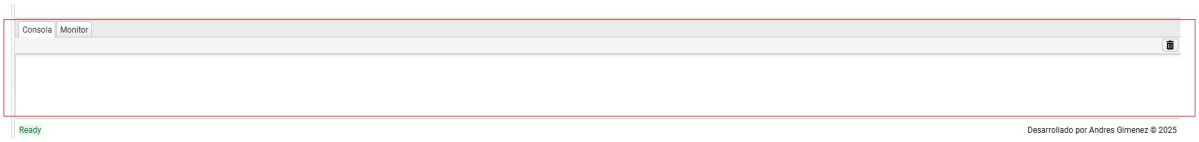

Figura A.5: Panel de consola y monitor.

Figura A.6: Barra de estado.

Este panel se divide en dos pestañas principales: **Consola** y **Monitor**, cada una con funciones específicas para la salida de compilación, ejecución y monitorización de la aplicación en tiempo real. Además, en la parte inferior de la ventana se encuentra la barra de estado que muestra el estado actual del editor.

A.4. Gestión de Proyectos

En esta sección se describen las herramientas para crear, abrir y organizar proyectos de programación en SamaruC Retro Editor.

SamaruC Retro Editor utiliza una estructura de proyecto basada en directorios, donde cada proyecto corresponde a una carpeta que contiene los archivos fuente (.c), encabezados (.h) y otros recursos relacionados. El panel del explorador de proyectos permite gestionar esta estructura de manera visual e intuitiva.

A.4.1. Crear un Nuevo Proyecto

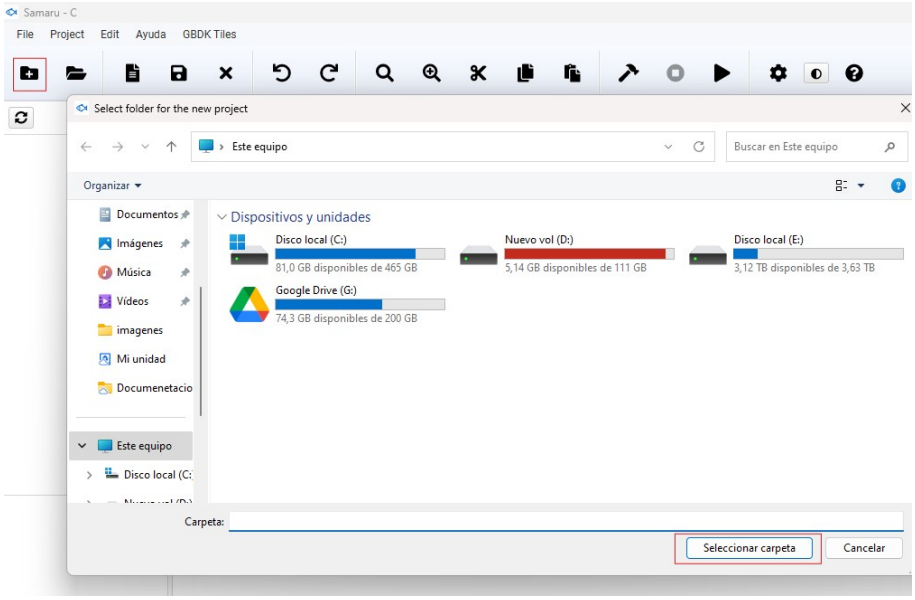

Figura A.7: Crear un nuevo proyecto.

1. Haz clic en el botón **Nuevo Proyecto** o usa el menú *Archivo* → *Nuevo Proyecto*
2. Selecciona o crea una carpeta para el proyecto
3. El explorador mostrará la estructura del directorio
4. Comienza a crear archivos .c y .h

A.4.2. Abrir un Proyecto Existente

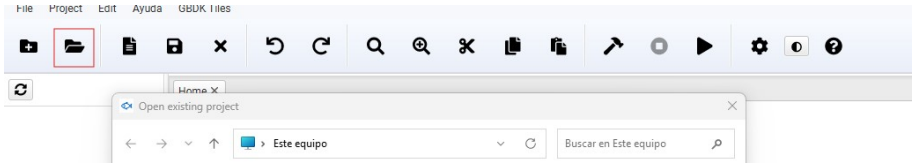

Figura A.8: Abrir un proyecto existente.

1. Haz clic en **Abrir Proyecto** o usa el menú *Archivo* → *Abrir Proyecto*
2. Navega hasta la carpeta del proyecto
3. Selecciona el directorio raíz del proyecto
4. Todos los archivos del proyecto aparecerán en el explorador

Consejo: Organiza tus proyectos con subdirectorios (src/, include/, build/) para mejor estructura y mantenimiento.

A.4.3. Trabajar con Archivos

SamaruC Retro Editor permite crear, abrir, guardar y cerrar archivos de manera sencilla desde el menú de archivos o la barra de herramientas. Las operaciones básicas incluyen:

- **Crear archivo nuevo:** Botón "Nuevo.º `Ctrl+N`
- **Abrir archivo:** Doble clic en el explorador o botón .ªbrir"
- **Guardar archivo:** `Ctrl+S` o botón "Guardar"
- **Guardar como:** Menú *Archivo* → *Guardar Como*
- **Cerrar archivo:** Botón Çerrar.º click en X de la pestaña

A.5. Edición de Código

En esta sección se describen las herramientas y características disponibles para la edición de código fuente en SamaruC Retro Editor, incluyendo el resaltado de sintaxis, operaciones de edición y características avanzadas del editor.

A.5.1. Resaltado de Sintaxis

El editor de texto integrado en SamaruC Retro Editor utiliza la biblioteca RichTextFX para proporcionar resaltado de sintaxis automático para el lenguaje C. Esto facilita la lectura y escritura de código al diferenciar visualmente los distintos elementos del lenguaje como se puede ver en la siguiente captura de pantalla.

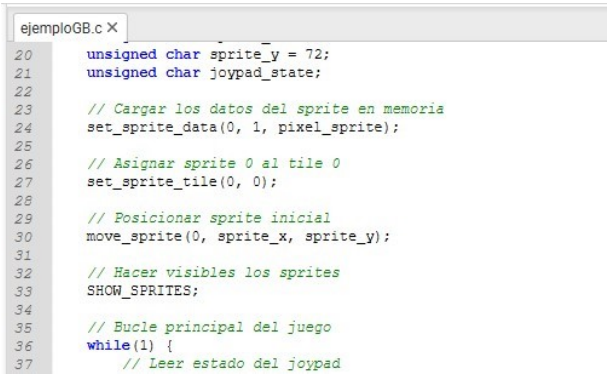

Figura A.9: Resaltado de sintaxis en el editor de código.

El editor proporciona resaltado automático para las siguientes categorías de elementos de código:

- **Palabras clave:** if, else, for, while, return, struct, etc.
- **Comentarios:** // comentarios de línea y /* bloques */
- **Cadenas:** ”texto entre comillas”
- **Números:** 123, 0xFF, 3.14
- **Funciones:** printf, scanf, main
- **Directivas:** #include, #define, #ifdef

A.5.2. Operaciones de Edición

En la siguiente tabla se resumen las operaciones de edición más comunes y sus atajos de teclado correspondientes para mejorar la eficiencia al escribir código:

Operación	**Atajo**	**Descripción**
Deshacer	`Ctrl+Z`	Revierte la última acción
Rehacer	`Ctrl+Y`	Restaura la acción deshecha
Cortar	`Ctrl+X`	Corta el texto seleccionado
Copiar	`Ctrl+C`	Copia el texto seleccionado
Pegar	`Ctrl+V`	Pega desde el portapapeles
Seleccionar todo	`Ctrl+A`	Selecciona todo el texto

A.5.3. Características Avanzadas

SamaruC Retro Editor incluye varias características avanzadas para mejorar la experiencia de edición de código y la productividad del programador:

- **Numeración de líneas:** Siempre visible para referencia
- **Múltiples pestañas:** Trabaja con varios archivos simultáneamente
- **Detección de cambios:** Asterisco (*) indica archivos modificados no guardados
- **Historial ilimitado:** Deshacer/Rehacer sin límites

A.6. Búsqueda de Texto

SamaruC Retro Editor ofrece potentes herramientas de búsqueda para localizar texto dentro de un archivo o en todo el proyecto. Estas funciones son esenciales para navegar rápidamente por el código, encontrar definiciones de funciones, referencias a variables o cualquier término específico.

A.6.1. Buscar en Archivo Actual

Cuando deseas buscar un término dentro del archivo que estás editando, puedes utilizar la función de búsqueda integrada de la siguiente manera:

- Presiona `Ctrl+F` o haz clic en el botón de búsqueda
- Se abre un panel de búsqueda en la parte superior del editor
- Escribe el término a buscar
- Navega entre resultados con los botones Anterior/Siguiente

A.6.2. Buscar en Todo el Proyecto

Si necesitas encontrar un término en todos los archivos de tu proyecto, la función de búsqueda en proyecto te permite hacerlo de manera eficiente:

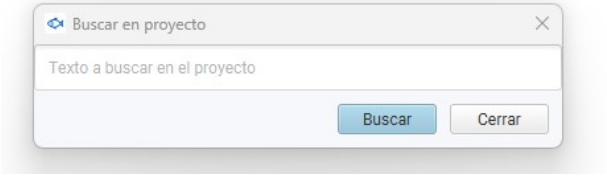

Figura A.10: Búsqueda en proyecto actual.

- Presiona `Ctrl+H` o usa el botón en la barra de herramientas
- Busca en todos los archivos del proyecto
- Los resultados muestran archivo, línea y contexto
- Haz doble clic en un resultado para abrir el archivo en esa ubicación

Figura A.11: Resultados de búsqueda en proyecto.

Consejo: Usa la búsqueda en proyecto para encontrar definiciones de funciones, referencias a variables o identificar dónde se usa una función específica.

A.7. Miscelánea

A.7.1. Atajos de Teclado

Para mejorar la eficiencia y la experiencia de usuario, el editor incluye una serie de atajos de teclado para las funciones más comunes. Estos atajos permiten realizar acciones rápidamente sin necesidad de navegar por los menús, lo que agiliza el flujo de trabajo. A continuación se presenta una tabla con los atajos de teclado disponibles en **SamaruC Retro Editor**:

Atajo	**Acción**
`Ctrl+N`	Nuevo archivo
`Ctrl+O`	Abrir archivo
`Ctrl+S`	Guardar archivo
`Ctrl+W`	Cerrar archivo actual
`Ctrl+Z`	Deshacer
`Ctrl+Y`	Rehacer
`Ctrl+X`	Cortar
`Ctrl+C`	Copiar
`Ctrl+V`	Pegar
`Ctrl+A`	Seleccionar todo
`Ctrl+F`	Buscar en archivo actual
`Ctrl+H`	Buscar en proyecto completo
`F5`	Compilar proyecto
`F6`	Ejecutar programa

Tabla A.1: Atajos de teclado disponibles en SamaruC Retro Editor.

A.7.2. Tecnologías utilizadas en el desarrollo de SamaruC Retro Editor:

- **JavaFX 17:** Framework de interfaz de usuario
- **RichTextFX:** Editor de texto con resaltado de sintaxis
- **Ikonli FontAwesome 5:** Iconos de interfaz
- **Maven:** Gestión de dependencias y construcción

A.7.3. Requisitos del Sistema:

- Java 17+
- 4 GB RAM mínimo (8 GB recomendado)
- 200 MB espacio en disco
- Compilador C (Z88DK o GBDK)

Apéndice B

Desarrollo de plugins

B.1. Introducción

SamaruC Editor incluye un sistema de plugins basado en **Java ServiceLoader** que permite extender la aplicación sin tocar su código fuente. Este documento explica paso a paso cómo crear, empaquetar e instalar un plugin.

B.2. Arquitectura general

La arquitectura de plugins se basa en una interfaz común `Plugin` que los desarrolladores implementan. SamaruC descubre e instancia estas clases automáticamente al arrancar, llamando a sus métodos de ciclo de vida para integrarlas en la aplicación, en la siguiente figura podemos ver un diagrama de la arquitectura del sistema de plugins.

```
+---------------------------------------------+
|            SamaruC Editor                   |
|                                             |
|  PluginManager                              |
|   +-- escanea carpeta plugins/              |
|   +-- carga cada JAR con URLClassLoader     |
|   +-- descubre implementaciones via         |
|   |   ServiceLoader<Plugin>                 |
|   +-- llama a onLoad -> onEnable            |
|                                             |
|  PluginContext  (pasado a cada plugin)      |
|   +-- getPrimaryStage()                     |
|   +-- getScene()                            |
|   +-- getMenuBar()                          |
|   +-- log(message)                          |
|   +-- registerMenu(menu)                    |
|   +-- unregisterMenu(menu)                  |
+---------------------------------------------+
         ^
         |  implements
+-----------------+
|   Tu plugin     |
|  (Plugin.java)  |
+-----------------+
```

Figura B.1: Diagrama de arquitectura del sistema de plugins.

El ciclo de vida de un plugin se describe en la siguiente tabla:

Método	**Cuándo se llama**
`onLoad(ctx)`	Justo después de cargar el JAR. Guarda el contexto aquí.
`onEnable()`	Cuando la UI está lista. Agrega tus menús/componentes aquí.
`onDisable()`	Cuando el plugin se desactiva o la app cierra. Limpia recursos aquí.
`getName()`	En cualquier momento. Devuelve el nombre visible del plugin.

Tabla B.1: Ciclo de vida de un plugin

B.3. Requisitos previos

- **JDK 17** o superior
- **Maven 3.8+** (o Gradle si lo prefieres)
- El JAR principal de SamaruC (`retro-editor-X.Y-SNAPSHOT.jar`) para compilar contra sus interfaces

Consejo: Añade el JAR de SamaruC a tu repositorio Maven local con:

```
mvn install:install-file -Dfile=retro-editor-1.0-SNAPSHOT.jar \
  -DgroupId=com.retroeditor -DartifactId=retro-editor \
  -Dversion=1.0-SNAPSHOT -Dpackaging=jar
```

B.4. Crear el proyecto Maven del plugin

Crea la siguiente estructura de directorios:

```
mi-plugin/
+-- pom.xml
+-- src/
    +-- main/
        +-- java/
        |   +-- com/
        |       +-- miplugin/
        |           +-- MiPlugin.java
        +-- resources/
            +-- META-INF/
                +-- services/
                    +-- com.retroeditor.plugin.Plugin
```

Figura B.2: Estructura de directorios del proyecto Maven del plugin.

`pom.xml` mínimo:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
             http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.miplugin</groupId>
    <artifactId>mi-plugin</artifactId>
    <version>1.0.0</version>
    <packaging>jar</packaging>

    <dependencies>
        <!-- Interfaces de SamaruC (solo compile-time) -->
        <dependency>
            <groupId>com.retroeditor</groupId>
            <artifactId>retro-editor</artifactId>
            <version>1.0-SNAPSHOT</version>
            <scope>provided</scope>
        </dependency>

        <!-- JavaFX (solo compile-time, lo provee el host) -->
        <dependency>
            <groupId>org.openjfx</groupId>
            <artifactId>javafx-controls</artifactId>
            <version>17.0.10</version>
            <scope>provided</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <version>3.8.1</version>
```

```
                <configuration>
                    <source>17</source>
                    <target>17</target>
                </configuration>
            </plugin>
        </plugins>
    </build>
</project>
```

Listing B.1: Configuración Maven básica

Importante: Las dependencias de SamaruC y JavaFX deben declararse con `<scope>provided</scope>` porque ya están disponibles en el classpath del host en tiempo de ejecución.

B.5. Implementar la interfaz `Plugin`

Crea `MiPlugin.java` implementando `com.retroeditor.plugin.Plugin`:

```java
package com.miplugin;

import com.retroeditor.plugin.Plugin;
import com.retroeditor.plugin.PluginContext;
import javafx.application.Platform;
import javafx.scene.control.Menu;
import javafx.scene.control.MenuItem;

public class MiPlugin implements Plugin {

    private PluginContext context;
    private Menu          miMenu;

    /** Se llama al cargar el JAR. Guarda el contexto para usarlo despues. */
    @Override
    public void onLoad(PluginContext context) {
        this.context = context;
        context.log("MiPlugin cargado correctamente.");
    }

    /** Se llama cuando la UI esta lista. Aqui agregamos nuestros elementos. */
    @Override
    public void onEnable() {
        // Toda modificacion de la UI debe hacerse en el hilo de JavaFX
        Platform.runLater(() -> {
            miMenu = new Menu("Mi Plugin");

            MenuItem saludar = new MenuItem("Saludar");
            saludar.setOnAction(e -> context.log("Hola desde MiPlugin!"));

            miMenu.getItems().add(saludar);
```

```
32
33              // Registrar el menu: SamaruC lo eliminara automaticamente
34              // al desactivar el plugin
35              context.registerMenu(miMenu);
36          });
37      }
38
39      /** Se llama al deshabilitar o cerrar la app. Limpia tus recursos.
        */
40      @Override
41      public void onDisable() {
42          // Los menus registrados con registerMenu() se limpian
        automaticamente,
43          // pero aqui puedes liberar cualquier otro recurso propio.
44          context.log("MiPlugin desactivado.");
45      }
46
47      /** Nombre visible en la pantalla de configuracion de SamaruC. */
48      @Override
49      public String getName() {
50          return "Mi Plugin de Ejemplo";
51      }
52  }
```

Listing B.2: Implementación básica de un plugin

B.6. Registrar el plugin con ServiceLoader

Java ServiceLoader requiere un fichero descriptor en `META-INF/services/`.

Crea el archivo: `src/main/resources/META-INF/services/com.retroeditor.plugin.Plugin`

Con el contenido (nombre completo de tu clase implementadora):

```
1  com.miplugin.MiPlugin
```

Si tu plugin tiene **varias implementaciones**, ponlas una por línea:

```
1  com.miplugin.MiPlugin
2  com.miplugin.OtroPlugin
```

B.7. Usar el `PluginContext`

La interfaz `PluginContext` proporciona los siguientes métodos:

Método	Descripción
`getPrimaryStage()`	Devuelve el `Stage` principal de la aplicación. Útil para crear diálogos modales.
`getScene()`	Devuelve la `Scene` principal. Permite acceder al `root` para añadir nodos.
`getMenuBar()`	Devuelve la `MenuBar` de la app. Puedes añadir menús directamente o usar `registerMenu()`.
`log(String)`	Escribe un mensaje en la pestaña **Monitor** de SamaruC.
`registerMenu(Menu)`	Añade un `Menu` a la barra de menú y registra la referencia para limpieza automática.
`unregisterMenu(Menu)`	Elimina manualmente un menú registrado.
`unregisterAll()`	Elimina todos los elementos registrados por este plugin. Lo llama SamaruC automáticamente.

Tabla B.2: Métodos de PluginContext

B.7.1. Ejemplo: abrir un diálogo con el Stage del host

```
@Override
public void onEnable() {
    Platform.runLater(() -> {
        Menu menu = new Menu("Herramientas");
        MenuItem abrirDialogo = new MenuItem("Abrir mi dialogo");
        abrirDialogo.setOnAction(e -> {
            Stage dialogo = new Stage();
            dialogo.initOwner(context.getPrimaryStage());   // modal
   respecto al host
            dialogo.setTitle("Mi herramienta");
            // ... configurar la escena del dialogo ...
            dialogo.show();
        });
        menu.getItems().add(abrirDialogo);
        context.registerMenu(menu);
    });
}
```

B.7.2. Ejemplo: escribir en el Monitor

```
context.log("Procesando fichero: " + archivo.getName());
```

B.8. Empaquetar el plugin como JAR

Desde la raíz de tu proyecto de plugin, ejecuta:

```
mvn package
```

Esto generará `target/mi-plugin-1.0.0.jar`. Ese es el fichero que debes instalar en SamaruC.

Nota: No uses el plugin `maven-shade` en el JAR del plugin para no duplicar clases del host (JavaFX, etc.). El JAR debe contener **solo** las clases propias del plugin.

B.9. Instalar el plugin en SamaruC

Tienes dos formas de instalar un plugin:

B.9.1. Opción A — Desde la interfaz de SamaruC (recomendado)

1. Abre SamaruC Editor.
2. Ve a **Archivo / Configuración** (o el atajo de teclado correspondiente).
3. Selecciona la pestaña **Plugins**.
4. Haz clic en **Instalar plugin** y selecciona tu fichero `.jar`.
5. SamaruC copiara el JAR a la carpeta `plugins/` junto al ejecutable.
6. Pulsa **Recargar plugins** para activarlo sin reiniciar.

B.9.2. Opción B — Copiado manual

Copia el fichero `.jar` directamente en la carpeta `plugins/` que se encuentra en el directorio de trabajo de SamaruC (al lado del ejecutable `RetroEditor.exe` o del JAR principal):

```
RetroEditor/
+-- RetroEditor.exe
+-- app/
|   +-- retro-editor-1.0-SNAPSHOT.jar
+-- plugins/              <- aqui va tu JAR
    +-- mi-plugin-1.0.0.jar
```

Reinicia la aplicación y el plugin se cargará automáticamente.

B.10. Habilitar y deshabilitar plugins

En la pestaña **Plugins** de la pantalla de configuración puedes:

- Ver la lista de todos los JARs descubiertos en `plugins/`.
- Ver su estado (habilitado / deshabilitado).
- Pulsar **Habilitar** o **Deshabilitar** para cambiar el estado.
- Pulsar **Recargar plugins** para aplicar cambios sin reiniciar.

El estado de cada plugin se guarda en el fichero:

```
~/.retroeditor.plugins.properties
```

(donde ~ es tu carpeta de usuario del sistema operativo)

B.11. Ejemplo completo

A continuación se muestra un plugin mínimo funcional que añade un menú **"Utilidades"** con una opción que muestra la ruta del archivo actualmente abierto.

B.11.1. Estructura del proyecto

```
plugin-utilidades/
+-- pom.xml
+-- src/main/
    +-- java/com/ejemplo/UtilsPlugin.java
    +-- resources/META-INF/services/com.retroeditor.plugin.Plugin
```

B.11.2. UtilsPlugin.java

```java
package com.ejemplo;

import com.retroeditor.plugin.Plugin;
import com.retroeditor.plugin.PluginContext;
import javafx.application.Platform;
import javafx.scene.control.*;

public class UtilsPlugin implements Plugin {

    private PluginContext ctx;

    @Override
    public void onLoad(PluginContext context) {
        this.ctx = context;
    }

    @Override
    public void onEnable() {
        Platform.runLater(() -> {
            Menu menuUtils = new Menu("Utilidades");

            MenuItem infoItem = new MenuItem("Informacion del entorno");
            infoItem.setOnAction(e -> {
                String info = "Java: "    + System.getProperty("java.version")
                            + "\nOS: "   + System.getProperty("os.name")
                            + "\nDir: "  + System.getProperty("user.dir");
                ctx.log(info);

                Alert alert = new Alert(Alert.AlertType.INFORMATION, info, ButtonType.OK);
```

```
                alert.setTitle("Entorno");
                alert.initOwner(ctx.getPrimaryStage());
                alert.showAndWait();
            });

            menuUtils.getItems().add(infoItem);
            ctx.registerMenu(menuUtils);
        });
    }

    @Override
    public void onDisable() {
        ctx.log("UtilsPlugin desactivado.");
    }

    @Override
    public String getName() {
        return "Plugin de Utilidades";
    }
}
```

B.11.3. META-INF/services/com.retroeditor.plugin.Plugin

```
com.ejemplo.UtilsPlugin
```

B.11.4. Compilar e instalar

```
cd plugin-utilidades
mvn package
copy target\plugin-utilidades-1.0.0.jar ..\RetroEditor\plugins\
```

B.12. Buenas prácticas y limitaciones

B.12.1. Buenas prácticas

- **Siempre usa** `Platform.runLater()` para cualquier modificación de la UI de JavaFX desde métodos del ciclo de vida del plugin.
- **Limpia tus recursos en** `onDisable()`: cierra hilos, streams, conexiones de red, etc.
- **Usa** `context.registerMenu()` en lugar de acceder directamente a `getMenuBar()` `.getMenus().add()` para que SamaruC pueda limpiar automáticamente al desactivar el plugin.
- **Declara** `provided` todas las dependencias que ya provee el host (JavaFX, SamaruC, etc.).
- **Versiona tu plugin** y sigue convenciones de nombre de paquete para evitar conflictos de clases.

B.12.2. Limitaciones actuales

- Los plugins **no pueden reemplazar** el editor de código ni el árbol de proyecto (no existen puntos de extensión para esos componentes todavía).
- No hay sistema de dependencias entre plugins: si tu plugin depende de otro, ambos JARs deben estar presentes y el orden de carga no está garantizado.
- El classloader usa estrategia **child-first**, por lo que puedes incluir versiones propias de librerías en tu JAR; sin embargo, las clases de `java.*`, `javax.*`, `javafx.*` y `org.kordamp.*` siempre se cargan desde el host.
- Los plugins se cargan **desde el directorio de trabajo** (`plugins/` junto al ejecutable), no desde rutas arbitrarias del sistema.
- Actualmente no existe una API para acceder al contenido del editor de texto abierto. Esto se planea para futuras versiones.